



**Department of Electronics & Telecommunication
Engineering**

**Laboratory Journal
For
MDM-331-ETC: Machine Learning Lab
SEMESTER – V**

NAME: _____

ROLL NO.: _____ **EXAM SEAT NO.:** _____

Pravara Rural Engineering College, Loni
A/P: Loni – 413 736, Tal. Rahata, Dist. Ahmednagar,
Maharashtra.

Vision of Institute

To provide quality technical education to foster youth with skills and values to make them capable of delivering significant contributions in local to global development.

Mission of Institute

- M1: Providing a conducive environment for effective teaching-learning, research, and innovation.
- M2: Establishing an ecosystem to nurture state-of-the-art skill sets through training, collaborating, and mentoring.
- M3: Ensuring a culture of continuous improvement, effective communication, and transparency among all stakeholders.
- M4: Promoting ethical and moral values to develop responsible and socially conscious citizens.

Vision of Department

To develop technical ability in Electronics and Telecommunication Engineering graduates by providing quality education that creates professionals for the benefit of society.

Mission of Department

To educate students in the advanced technologies in the field of Electronics and Telecommunication for emphasizing technical expertise, professional attitude, ethical values and inspire the students to utilize their education for the betterment of society.

- M1: To educate students in the advanced technologies in the field of Electronics and Telecommunication Engineering.
- M2: To develop technical expertise, professional attitude and ethical practices.
- M3: To inspire the students to utilize their education for the betterment of society.

Program Educational Objectives (PEOs)

Graduates would demonstrate ability to:

- PEO1: Engage in Designing, Testing, Operating, Manufacturing Systems in the field of Electronics and Telecommunication Engineering.
- PEO2: Solve Problems of allied areas by applying the knowledge of Electronics and Telecommunication Engineering.
- PEO3: Ability to update knowledge with emerging technologies to embrace Professional and ethical attitude in multidisciplinary projects.
- PEO4: Work effectively as individual and as team member to make effective contributions to the benefit of workplace and community.

Program Specific Outcomes (PSOs)

After successful completion of Degree our graduates will be able to:

- PSO1: Ability to exhibit competency in the areas of Electronics and Telecommunication Engineering like Electronic Circuits and Communication.
- PSO2: Design and implement the Embedded systems to resolve societal and industrial problems using modern engineering hardware and software tools.
- PSO3: Ability to inculcate software proficiency skills for Industry.

Knowledge and Attitude Profile (WK)

A Knowledge and Attitude Profile (KAP), often represented as WK (Knowledge and Attitude Profile) in some contexts, is a framework or assessment tool used to evaluate an individual's knowledge and attitudes related to a specific area, topic, or domain.

Code	Description
WK1	A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.
WK2	Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.
WK3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
WK4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK5	Knowledge, including efficient resource use, environmental impacts, whole-life cost, re-use of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.
WK6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
WK7	Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.

WK8	Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.
WK9	Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

Programme Outcomes (PO)

Program Outcomes are statements that describe what students are expected to know and be able to do upon graduating from the program. These relate to the skills, knowledge, attitude and behaviour that students acquire through the program. On successful completion of B.E. in Electronics and Telecommunication Engineering, graduating students/graduates will be able to:

Code	Title	Description
PO1	Engineering Knowledge	Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop the solution of complex engineering problems.
PO2	Problem Analysis	Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/ Development of Solutions	Design creative solutions for complex engineering problems and design / develop systems / components /processes to meet identified needs with consideration for public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	Conduct Investigations of Complex Problems	Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)

PO5	Engineering Tool Usage	Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling, recognizing their limitations, to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and the World	Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)
PO7	Ethics	Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team Work	Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication	Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
PO10	Project Management and Finance	Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects in multidisciplinary environments.
PO11	Life-Long Learning	Recognize the need for, and have the preparation and ability for (i) independent and life-long learning, (ii) adaptability to new and emerging technologies, and (iii) critical thinking in the broadest context of technological change. (WK8)

Course Objectives

To make students understand:

- To introduce the fundamentals of Artificial Intelligence, Machine Learning, and data-driven systems.
- To develop students' ability to preprocess, analyze, and visualize engineering datasets.
- To enable students to implement Machine Learning algorithms using Python-based tools.
- Introduce emerging technologies such as Edge AI, TinyML, AIoT, and Generative AI.
- Prepare students for industry-oriented AI projects, internships, research, and multidisciplinary applications.

Course Outcomes (COs)

After completion of the course, students will be able to:

- **CO1:** Explain Machine Learning fundamentals, workflow and AI applications in Electronics and Telecommunication Engineering.
- **CO2:** Apply data preprocessing, visualization, and exploratory analysis techniques using Python libraries.
- **CO3:** Implement Supervised Machine Learning Algorithms for prediction and classification problems.
- **CO4:** Apply Unsupervised Learning techniques and analyze engineering datasets using clustering approaches.
- **CO5:** Develop AI-based mini-projects using modern tools, real-world datasets and emerging AI technologies.

Pravara Rural Engineering College, Loni
Department of Electronics and Telecommunication Engineering
Subject: Machine Learning Lab
Index

Sr. No	Date	Title of Experiment	Pages	Marks	Signature
1.		Perform installation and familiarization of Python programming environment using Google Colab.			
2.		Visualize engineering datasets using graphs, histograms, and scatter plots with Matplotlib.			
3.		Perform data preprocessing and cleaning operations on real-world datasets.			
4.		Apply feature scaling, normalization, and standardization techniques using Python tools.			
5.		Implement Linear Regression algorithm for prediction of engineering parameters.			
6.		Develop classification models using Logistic Regression for real-world datasets.			
7.		Implement K-Means clustering algorithm for grouping and pattern analysis of datasets.			
8.		Demonstrate dimensionality reduction using Principal Component Analysis (PCA).			
9.		Develop and implement a mini-project using Machine Learning techniques and real-world datasets.			

This is to certify that Mr./ Ms. _____ a student of T.E. E&TC has performed the above experiments in the _____ Laboratory (ETC Dept.) for Machine learning Lab subject, at the Pravara Rural Engineering College, Loni.

STAFF INCHARGE

DATE

HEAD OF DEPT.

Experiment No. 1

Perform Installation and Familiarization of Python Programming Environment using Google Colab

1. Aim

To install, access, and become familiar with the Python programming environment using Google Colab, and to execute basic Python programs and understand the Colab interface.

2. Objectives

After completing this experiment, students will be able to:

- Understand the Python programming environment.
- Access and use Google Colab for Python programming.
- Create and execute Python code in Colab.
- Understand code cells, text cells, and the output area.
- Perform basic Python operations.
- Import and use Python libraries.
- Upload and access files in Google Colab.
- Save and share a Colab notebook.

3. Software and Tools Required

Sr. No.	Software/Tool	Purpose
1	Computer/Laptop	Running the experiment
2	Internet Connection	Accessing Google Colab
3	Web Browser	Opening Colab
4	Google Account	Creating/saving notebooks
5	Google Colab	Python programming environment

4. Theory

4.1 Introduction to Python

Python is a high-level, interpreted, general-purpose programming language created with an emphasis on code readability and simplicity. Unlike compiled languages such as C or C++, Python code is executed line by line by an interpreter, which makes it easier to test and debug small pieces

of logic interactively — a property that is especially useful in engineering experimentation and data analysis, where code is often written, tested, and modified incrementally.

Python is widely used in Artificial Intelligence, Machine Learning, Deep Learning, Data Science, Signal Processing, Computer Vision, Web Development, Scientific Computing, and the Internet of Things (IoT). Python is particularly popular in machine learning because it provides a large collection of libraries such as NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, and PyTorch.

4.2 Why Python for Engineering and Machine Learning?

- Simple, readable syntax that reduces development time compared to lower-level languages.
- A vast ecosystem of open-source scientific and engineering libraries (NumPy, SciPy, Pandas).
- Strong community support, extensive documentation, and ready-made code examples.
- Cross-platform compatibility — the same code runs on Windows, Linux, and macOS.
- Seamless integration with cloud platforms, databases, and hardware interfaces (e.g., sensors, microcontrollers), which is valuable for IoT-based engineering datasets such as the one used in this laboratory course.

4.3 What is Google Colab?

Google Colaboratory (Google Colab) is a cloud-based environment that allows users to write and execute Python programs through a web browser. It provides a Jupyter Notebook-like interface and does not require a complete Python installation on the student's computer for basic use. Colab notebooks run on virtual machines hosted on Google's infrastructure, so the same notebook can be accessed, edited, and executed from any device with a browser and an internet connection.

A Colab notebook consists mainly of:

- Code cells
- Text/Markdown cells
- Output areas
- Files
- Runtime environment

4.4 Advantages of Google Colab

- No local Python installation is required for basic programming.
- Programs can be executed through a web browser.
- Python libraries can be imported and used easily.
- Notebooks can be saved online.
- Files can be uploaded and processed.
- Notebooks can be shared with other users.

- It provides access to cloud computing resources, including free access to GPUs/TPUs for computationally heavy tasks.
- It is useful for machine learning and data science experiments.

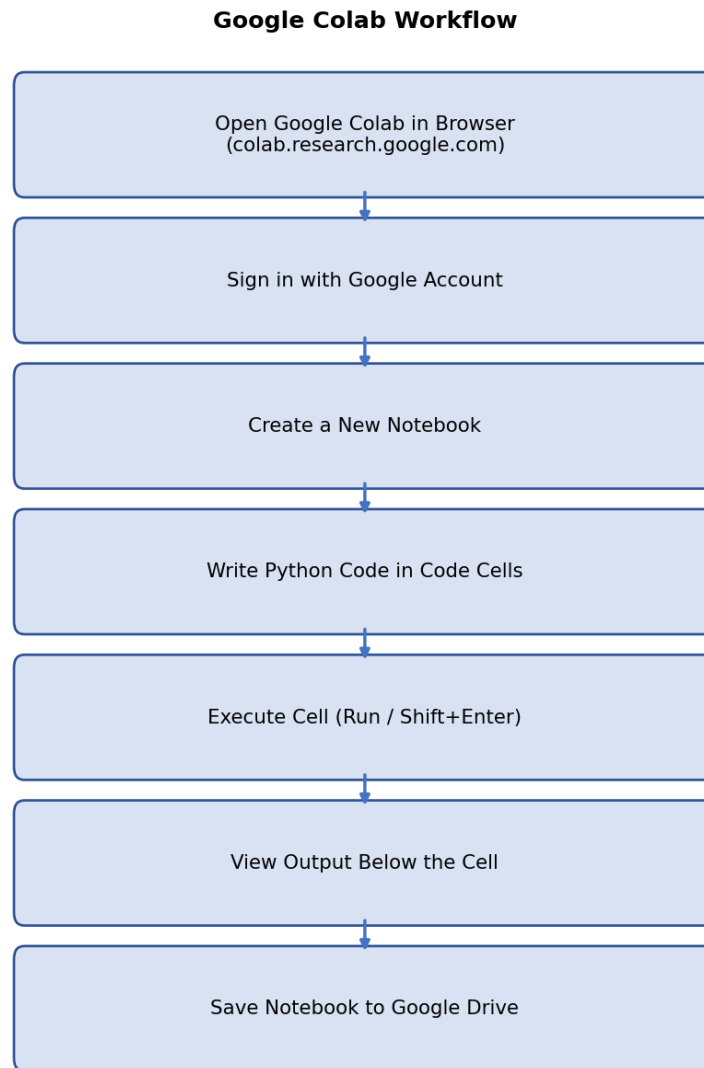


Fig. 1.1 — Typical workflow for creating and running a program in Google Colab

4.5 Google Colab Interface

A typical Colab notebook contains the following components:

- Code Cells — used to write and execute Python programs; the output is displayed below the code cell.
- Text/Markdown Cells — used to write instructions, explanations, headings, notes, and documentation.
- Execution — code is executed using the Run button or Shift + Enter.

- Output — the result generated after execution is displayed below the corresponding code cell.
- Runtime Menu — used to start, restart, interrupt, or change the type of runtime (CPU/GPU/TPU).
- Files Panel — a temporary file browser for the notebook's virtual machine, used to upload and inspect datasets such as CSV files.

4.6 The Python Machine Learning Library Ecosystem

This laboratory course relies on a small set of libraries that form the foundation of almost every Python-based Machine Learning workflow. NumPy provides fast array-based numerical computation; Pandas provides labelled, tabular data structures (DataFrames) for loading and manipulating datasets such as the Telecom Tower IoT dataset; Matplotlib provides plotting and visualization; and Scikit-learn provides ready-made implementations of standard Machine Learning algorithms, preprocessing tools, and evaluation metrics, used extensively from Experiment No. 3 onward.

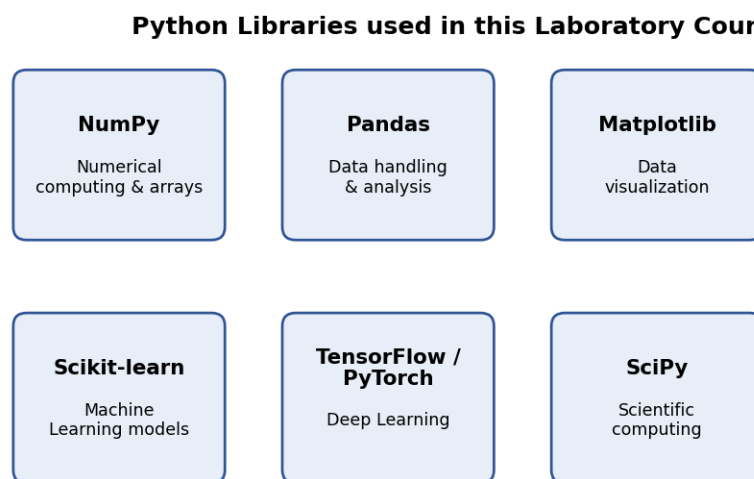


Fig. 1.2 — Core Python libraries used throughout this laboratory course

5. Procedure

Step 1: Open Google Colab

Open a web browser such as Google Chrome or Microsoft Edge and enter the Google Colab URL: <https://colab.research.google.com/>. The Google Colab welcome page will appear.

Step 2: Sign in to Google Account

If required, click Sign in, enter the Google account credentials, and after successful login the Colab interface will open. A Google account is useful for saving notebooks to Google Drive.

Step 3: Create a New Notebook

On the Colab welcome page, select New Notebook. A new notebook will be created containing a code cell. The notebook can be renamed by clicking the notebook name at the top — for example, rename Untitled0.ipynb to Experiment_01_Python_Colab.ipynb.

Step 4: Understanding Code Cells

A code cell is an area in which Python programs are written and executed. Enter the following program and execute the cell using the Run button:

```
print("Hello World")
```

Output:

```
Hello World
```

Step 5: Creating a Text/Markdown Cell

Text cells are used to add experiment titles, theory, instructions, explanations, and notes. Click + Text and enter:

```
# Introduction to Python
```

Step 6: Python Comments

Comments are used to explain Python code. Example of a single-line comment:

```
# This is a comment
```

Step 7: Basic Python Operations

```
a = 20
b = 10
print("Addition =", a + b)
print("Subtraction =", a - b)
print("Multiplication =", a * b)
print("Division =", a / b)
```

Output:

```
Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2.0
```

Step 8: Python Variables

A variable stores a value. Example:

```

name = "Student"
age = 20
marks = 85.5

print("Name:", name)
print("Age:", age)
print("Marks:", marks)

```

Step 9: Python Data Types

Python supports several basic data types:

Data Type	Example
Integer	10
Float	10.5
String	"Python"
Boolean	True
List	[10, 20, 30]

```

x = 10
y = 10.5
name = "Python"
status = True
print(type(x))
print(type(y))
print(type(name))
print(type(status))

```

Step 10: Python Lists

A list stores multiple values.

```

marks = [75, 82, 68, 91, 88]
print("Marks:", marks)
print("First mark:", marks[0])
print("Number of marks:", len(marks))

```

Step 11: Conditional Statements

Python uses conditional statements to make decisions.

```

marks = 75
if marks >= 40:

```

```
print("Pass")
else:
    print("Fail")
```

Output:

```
Pass
```

Step 12: Loops

A loop is used to execute a set of statements repeatedly. Example — For Loop:

```
for i in range(1, 6):
    print(i)
```

Step 13: Functions

A function is a reusable block of code.

```
def addition(a, b):
    return a + b

result = addition(10, 20)
print("Result =", result)
```

Output:

```
Result = 30
```

Step 14: Installing/Using Python Libraries

One of the important features of Colab is the ability to use Python libraries that are commonly used in machine learning experiments. For example:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Step 15: NumPy

NumPy is a Python library used for numerical computing and array operations.

```
import numpy as np
data = np.array([10, 20, 30, 40, 50])
print("Array:", data)
print("Mean:", np.mean(data))
print("Maximum:", np.max(data))
print("Minimum:", np.min(data))
```

Step 16: Pandas

Pandas is a Python library used for data manipulation and analysis.

```
import pandas as pd
data = {
    "Name": ["Amit", "Neha", "Rahul"],
    "Marks": [80, 90, 75]
}
df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Marks
0	Amit	80
1	Neha	90
2	Rahul	75

Step 17: Matplotlib

Matplotlib is used for data visualization.

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]
plt.plot(x, y)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Simple Line Graph")
plt.grid(True)
plt.show()
```

Step 18: Installing an Additional Python Package

Some packages may not already be available in the runtime environment. They can be installed using:

```
!pip install package_name

# Example
!pip install openpyxl
```

After installation, the package can be imported normally:

```
import openpyxl
```

Note: Many commonly used machine learning libraries are already available in the Colab environment. Installation is therefore only required when a required package is not already available or a particular version is needed.

Step 19: Runtime Environment

Google Colab executes Python code using a temporary cloud-based runtime. Students can check the Python version using:

```
import sys
print("Python Version:")
print(sys.version)
```

Step 20: Checking Installed Libraries

Students can check the versions of commonly used libraries.

```
import numpy
print("NumPy Version:", numpy.__version__)
```

Step 21: Saving the Notebook

The notebook can be saved in Google Drive. The notebook file generally has the extension .ipynb, for example Experiment_01_Python_Colab.ipynb. Students should save the notebook using a meaningful name.

Step 22: Sharing the Notebook

A Colab notebook can be shared using the Share option available in the Colab interface. Before sharing, students should verify the appropriate access permissions.

6. Result

The Python programming environment was successfully accessed and familiarized using Google Colab. Basic Python programs were executed, and the use of NumPy, Pandas, and Matplotlib libraries was demonstrated.

7. Conclusion:

Date of Performance		Date of Submission	
Assessment Rubrics	Attendance	Performance (4)	Completion (3) Total (10)
Marks Obtained			
Teacher's Signature			

8. Viva Voce Questions

- Q1. What type of programming language is Python — compiled or interpreted?
- Q2. What is Google Colab?
- Q3. What are the two main types of cells in a Colab notebook?
- Q4. How is a code cell executed in Colab?
- Q5. Name any three Python libraries commonly used in Machine Learning.
- Q6. What command is used to install a Python package that is not already available in Colab?
- Q7. What is the difference between a list and a variable in Python?
- Q8. What is the purpose of a comment in Python code?
- Q9. What extension is used to save a Colab notebook?
- Q10. Why are cloud-based environments like Colab useful for Machine Learning experiments?

Experiment No. 2

Visualize Engineering Datasets using Graphs, Histograms and Scatter Plots with Matplotlib

1. Aim

To visualize engineering datasets using graphs, histograms, and scatter plots with the Python Matplotlib library and interpret the patterns, distributions, and relationships present in the data.

2. Objectives

After completing this experiment, students will be able to:

- Load and inspect an engineering dataset using Python.
- Use Pandas to read and analyze dataset information.
- Create basic graphs using Matplotlib.
- Plot the distribution of engineering data using a histogram.
- Analyze the relationship between two variables using a scatter plot.
- Add appropriate titles, axis labels, legends, and grids to plots.
- Interpret engineering data from graphical representations.

3. Software and Tools Required

Sr. No.	Software/Tool	Purpose
1	Python	Programming language
2	Google Colab / Jupyter Notebook	Python programming environment
3	Pandas	Dataset handling and analysis
4	NumPy	Numerical computation
5	Matplotlib	Data visualization
6	CSV Dataset	Telecom_Tower_IoT_Dataset.csv

4. Dataset Description

4.1 Dataset Overview

The dataset used in this laboratory course, `Telecom_Tower_IoT_Dataset.csv`, represents operational data simulated from IoT-enabled telecom towers deployed across urban, semi-urban, and rural regions. Each telecom tower is equipped with multiple sensors that continuously monitor the health and performance of critical electronic and communication subsystems. The collected sensor data is transmitted to the Network Operations Center, where it is analyzed to monitor network performance.

4.2 Data Collection Scenario

Each telecom tower periodically transmits its operational status to the central monitoring system. Every transmission is recorded as a single observation in the dataset. Therefore:

- One row represents one observation collected from a telecom tower at a particular instant of time.
- One column represents one engineering parameter (feature) measured by the IoT monitoring system.

4.3 Dataset Specifications

Attribute	Description
Application Domain	Telecom Network Health Monitoring
Dataset Type	Structured Tabular Dataset
Source	Simulated IoT Sensor Data
Number of Records	120
Number of Features	15 (including target variable)
Number of Input Features	14
Target Variable	Failure
File Format	CSV
Collection Frequency	Periodic IoT Sensor Readings

4.4 Feature Description

Feature	Unit	Description	Data Type
Tower_ID	—	Unique identifier of telecom tower	Categorical

Timestamp	Date-Time	Date and time of data collection	Date-Time
Region	—	Location category (Urban / Semi-Urban / Rural)	Categorical
Temperature_C	°C	Internal equipment temperature	Numerical
Humidity_pct	%	Relative humidity	Numerical
Battery_Voltage_V	V	Backup battery voltage	Numerical
Current_A	A	Current consumed by tower	Numerical
CPU_Load_pct	%	Processor utilization of baseband	Numerical
Signal_Strength_dBm	dBm	Received signal strength	Numerical
Packet_Loss_pct	%	Percentage of lost packets	Numerical
Active_Users	Count	Number of users connected	Numerical
Power_Consumption_kW	kW	Total power consumed by the tower	Numerical
Cooling_Fan_RPM	RPM	Cooling fan rotational speed	Numerical
Maintenance_History	Count	Number of maintenance activities	Numerical
Failure	Yes / No	Indicates equipment failure status (Target Variable)	Categorical

4.5 Engineering Relationships Embedded in the Dataset

This dataset follows realistic engineering principles observed in telecom infrastructure. These relationships enable students to discover meaningful patterns through visualization and Machine Learning.

- Temperature ↑ → Battery Voltage ↓ — battery efficiency decreases as operating temperature increases.
- Temperature ↑ → Current Consumption ↑ — electronic components draw higher current under thermal stress.
- Temperature ↑ → CPU Load ↑ — higher processing activity generates additional heat.
- Temperature ↑ → Cooling Fan Speed ↑ — the cooling system increases fan speed to dissipate excess heat.
- Temperature ↑ → Signal Strength ↓ — elevated temperatures can adversely affect RF components.

- Temperature ↑ → Packet Loss ↑ — network performance may degrade under abnormal thermal conditions.
- Current Consumption ↑ and CPU Load ↑ → Power Consumption ↑.
- Multiple Abnormal Conditions → Equipment Failure = Yes.

Note: The dataset contains 120 records, 3 regions (40 towers each), a very small number of missing sensor readings, and a small number of failure cases, reflecting the fact that equipment failure is a relatively rare event in practice. Students are expected to investigate and verify these relationships using appropriate visualization techniques rather than assuming them beforehand.

4.6 Why Visualization Matters in Engineering Analysis

Numerical summaries such as the mean or standard deviation describe a dataset with a small number of values, but they can hide important structure — two datasets with identical means and standard deviations can have very different shapes, spreads, or outliers. Visualization exposes this structure directly to the eye, letting an engineer quickly spot skewed distributions, clusters, trends over time, and unusual readings before any formal model is built. Each plot type answers a different type of question, as summarized below.

Plot Type	Best Suited For	Typical Engineering Question
Histogram	Distribution of one numerical variable	What is the normal operating range?
Scatter Plot	Relationship between two numerical variables	Does X affect Y, and how strongly?
Bar Graph	Comparing a numerical value across categories	Which region consumes the most power?
Line Plot	Change of a variable over an ordered sequence (time)	Is CPU load trending up or down?
Box Plot	Spread, median, and outliers of a numerical variable	Are there any abnormal sensor readings?

5. Procedure

Investigation 1: Understanding the Dataset

Task 1: Import the Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Task 2: Load the Dataset

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
```

Task 3: Display the First Five Records

```
print(df.head())
```

Task 4: Display the Last Five Records

```
print(df.tail())
```

Task 5: Determine Dataset Dimensions

```
print(df.shape)
```

Task 6: Display Dataset Information

```
print(df.info())
```

Task 7: Generate Statistical Summary

```
print(df.describe())
```

Task 8: Display Column Names

```
print(df.columns)
```

Investigation 2: Visualizing the Distribution of Equipment Temperature using Histogram

A Histogram is used to visualize the frequency distribution of a continuous variable. It helps engineers answer questions such as: Which temperature range occurs most frequently? Are most towers operating within a safe temperature range? Are there unusually high or low temperatures? Is the distribution symmetric or skewed? Are there possible outliers?

Step 1: Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load the Dataset

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
```

Step 3: Plot the Histogram

```
plt.figure(figsize=(8,5))
plt.hist(df["Temperature_C"], bins=10, edgecolor="black")
plt.title("Distribution of Equipment Temperature")
plt.xlabel("Temperature (°C)")
plt.ylabel("Number of Towers")
```

```
plt.grid(alpha=0.3)
plt.show()
```

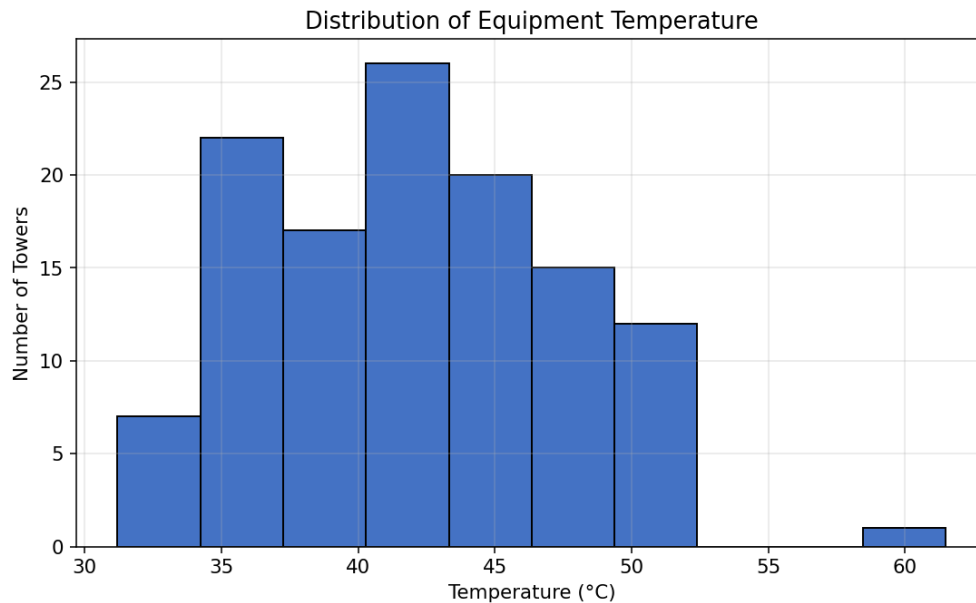


Fig. 2.1 — Actual histogram of equipment temperature generated from Telecom_Tower_IoT_Dataset.csv

Observed Output: The histogram is centred in the low-to-mid 40s °C, with progressively fewer towers as temperature moves toward either extreme of the range — typical of a roughly bell-shaped operating distribution with a mild right skew.

Investigation 3: Investigating the Relationship between Equipment Temperature and CPU Load using Scatter Plot

A Scatter Plot is used to study the relationship between two numerical variables. It helps engineers determine whether two parameters are related, whether the relationship is positive or negative, whether it appears linear, and whether any observations deviate significantly from the general trend.

Step 1: Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
```

Step 3: Generate Scatter Plot

```
plt.figure(figsize=(8,5))
plt.scatter(df["CPU_Load_pct"], df["Temperature_C"], alpha=0.8)
plt.title("CPU Load vs Equipment Temperature")
```

```
plt.xlabel("CPU Load (%)")
plt.ylabel("Temperature (°C)")
plt.grid(alpha=0.3)
plt.show()
```

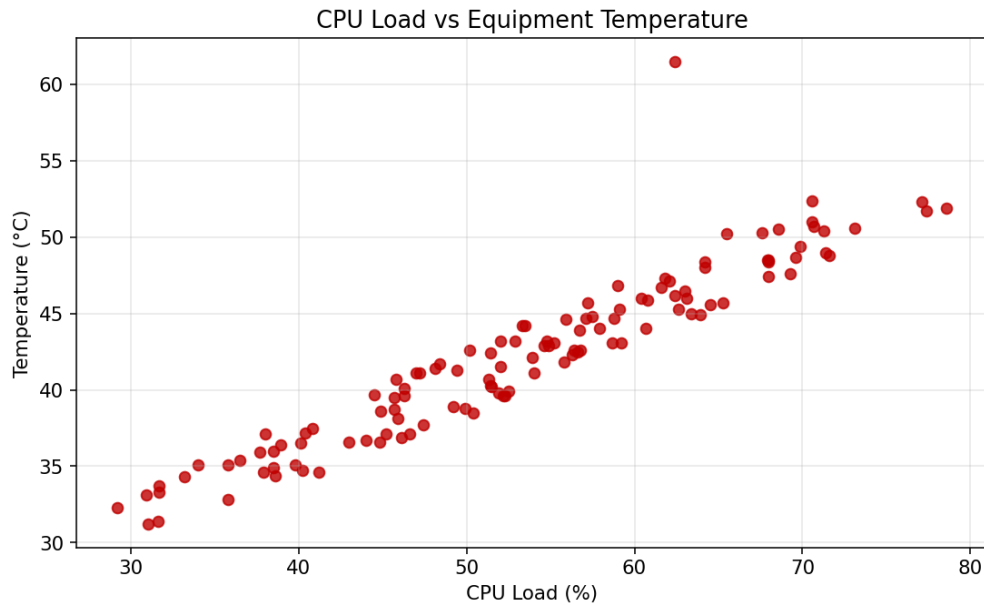


Fig. 2.2 — Actual scatter plot of CPU Load vs Equipment Temperature

Observed Output: The points trend upward from left to right — towers with higher CPU load consistently show higher equipment temperature — confirming the positive engineering relationship described in Section 4.5.

Investigation 4: Comparing Average Power Consumption across Different Regions using Bar Graph

A Bar Graph is used to compare numerical values across different categories. In this experiment, the category is Region (Urban, Semi-Urban, Rural) and the numerical value is Average Power Consumption.

Step 1: Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
```

Step 3: Calculate Average Power Consumption

```
avg_power = df.groupby("Region")["Power_Consumption_kW"].mean()
```

Step 4: Plot Bar Graph

```
plt.figure(figsize=(7,5))
avg_power.plot(kind="bar", edgecolor="black")
plt.title("Average Power Consumption by Region")
plt.xlabel("Region")
plt.ylabel("Average Power Consumption (kW)")
plt.grid(axis="y", alpha=0.3)
plt.show()
```

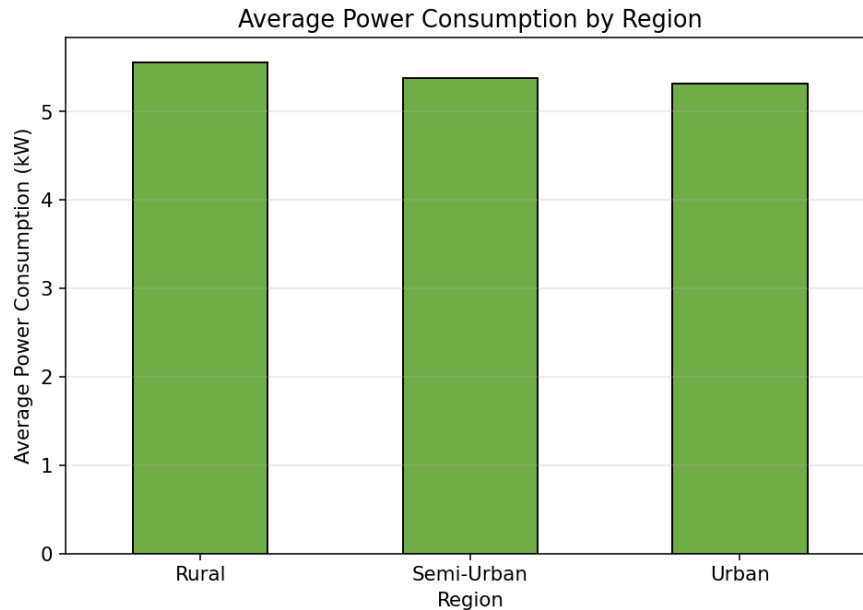


Fig. 2.3 — Actual bar graph of average power consumption by region

Investigation 5: Monitoring CPU Load Over Time using Line Plot

A Line Plot is used to visualize changes in a numerical variable over an ordered sequence, such as time. It helps engineers observe trends over time, detect sudden increases or decreases, identify recurring patterns, and monitor system performance continuously.

Step 1: Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load the Dataset

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
```

Step 3: Convert Timestamp to Date-Time Format

```
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
```

Step 4: Arrange Data Chronologically

```
df = df.sort_values("Timestamp")
```

Step 5: Generate Line Plot

```
plt.figure(figsize=(10,5))  
plt.plot(df["Timestamp"], df["CPU_Load_pct"], linewidth=2)  
plt.title("CPU Load Variation Over Time")  
plt.xlabel("Time")  
plt.ylabel("CPU Load (%)")  
plt.grid(alpha=0.3)  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

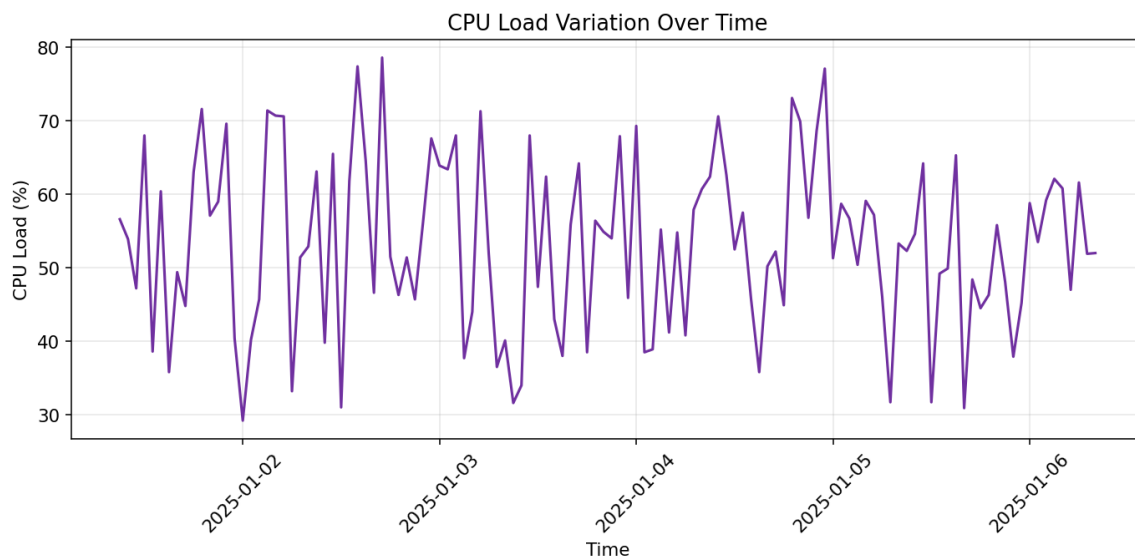


Fig. 2.4 — Actual line plot of CPU load variation over time

6. Result

The Telecom Tower IoT dataset was successfully loaded and visualized using histograms, scatter plots, bar graphs, and line plots. The plots confirmed the presence of a positive relationship between CPU load and equipment temperature, revealed differences in average power consumption across regions, and showed the time-varying nature of CPU load.

7. Conclusion:

Date of Performance			Date of Submission		
Assessment Rubrics	Attendance	Performance (4)	Completion (3)	Total (10)	
Marks Obtained					
Teacher's Signature					

8. Viva Voce Questions

Q1. What is a histogram used for?

Q2. What is the difference between a histogram and a bar graph?

Q3. What does a scatter plot show?

Q4. In the CPU Load vs Temperature scatter plot, what relationship was observed?

Q5. When would you use a line plot instead of a scatter plot?

Q6. Which Pandas function is used to load a CSV file into a DataFrame?

Q7. Which Pandas function returns summary statistics (mean, std, min, max, etc.) of numerical columns?

Q8. What does `df.info()` display?

Q9. Why is it necessary to convert the Timestamp column using `pd.to_datetime()`?

Q10. Based on this experiment, which region shows the highest average power consumption?

Experiment No. 3

Perform Data Preprocessing and Cleaning Operations on Real-World Datasets

1. Aim

To perform data preprocessing and cleaning operations on the Telecom Tower IoT dataset using Python and Pandas, including handling missing values, removing duplicate records, correcting inconsistent data, detecting and treating outliers, engineering new features, and encoding categorical variables.

2. Objectives

After completing this experiment, students will be able to:

- Load a real-world engineering dataset into Python.
- Understand the structure and characteristics of a dataset.
- Identify missing values and handle them appropriately.
- Detect and remove duplicate records.
- Identify and correct inconsistent categorical data.
- Detect and handle outliers using the IQR method.
- Perform feature selection and feature engineering.
- Encode categorical variables using Label Encoding and One-Hot Encoding.
- Save the cleaned dataset for further use in the laboratory course.

3. Software and Tools Required

Sr. No.	Software/Tool	Purpose
1	Python	Programming language
2	Google Colab / Jupyter Notebook	Python programming environment
3	Pandas	Dataset handling and analysis
4	NumPy	Numerical computation
5	Matplotlib	Data visualization (box plot)
6	Scikit-learn	Categorical encoding
7	CSV Dataset	Telecom_Tower_IoT_Dataset.csv

4. Theory

Data preprocessing is the process of converting raw data into a clean, consistent, and suitable form before applying machine learning algorithms. Real-world datasets frequently contain missing values, duplicate records, inconsistent formats, outliers, and categorical variables that require encoding. Therefore, preprocessing is an essential stage of the machine learning workflow, carried out before feature scaling (Experiment No. 4) and model building (Experiments No. 5 onward).

Note: This experiment focuses only on cleaning and structural preparation of the data. Feature scaling, normalization, and standardization — which adjust the numerical range of features — are covered separately in Experiment No. 4 and are not repeated here.

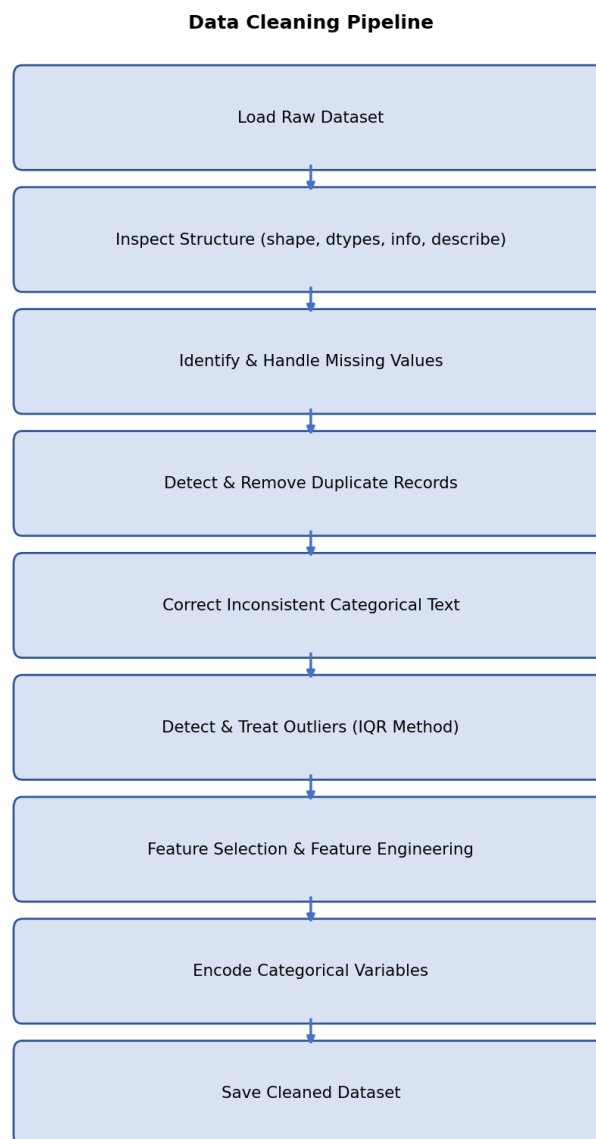


Fig. 3.1 — Data cleaning pipeline followed in this experiment

4.1 Why Real-World Data is “Dirty”

Sensor-driven datasets such as the Telecom Tower IoT dataset are collected automatically, over time, from many physical devices operating in uncontrolled field conditions. This introduces several categories of data-quality problems that a purely simulated or textbook dataset would not have:

- Missing readings — a sensor or communication link may briefly fail to report a value.
- Duplicate records — the same reading may be logged twice due to retransmission.
- Inconsistent categorical text — the same category may be entered with different capitalization or spacing (e.g., manual data entry, merged logs from different systems).
- Outliers — a faulty sensor, a genuine extreme weather event, or a data-entry slip can all produce values far outside the normal range.

Left uncorrected, these problems can bias statistical summaries, distort visualizations, and directly degrade the accuracy of any Machine Learning model trained on the data — which is why preprocessing is treated as its own dedicated stage of the workflow rather than an afterthought.

4.2 Types of Missing Data (for Context)

Type	Description
MCAR — Missing Completely at Random	The missingness is unrelated to any variable in the dataset (e.g., a random transmission drop).
MAR — Missing at Random	The missingness is related to another observed variable (e.g., readings more likely to be missing during a known network fault window).
MNAR — Missing Not at Random	The missingness is related to the missing value itself (e.g., a sensor stops reporting specifically because it has failed at an extreme reading).

Median imputation, used in this experiment, is a simple and reasonably robust default when the missingness is close to random and the proportion of missing values is small, as is the case here.

5. Procedure

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
```

Step 2: Load the Dataset

If the dataset is stored on the student's computer, upload it first in Google Colab:

```
from google.colab import files
uploaded = files.upload()
```

Then load the CSV file into a DataFrame:

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
```

Step 3: Display the Dataset

```
print(df.head()) # first five records
print(df.tail()) # last five records
print(df.sample(5)) # random sample
```

Step 4: Determine Dataset Dimensions

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

Expected Output: The dataset contains 120 rows (observations) and 15 columns (features), as described in Experiment No. 2.

Step 5: Display Column Names and Data Types

```
print(df.columns)
print(df.dtypes)
```

Numerical sensor readings (Temperature_C, Humidity_pct, Battery_Voltage_V, etc.) are stored as float64 or int64, while Tower_ID, Region, Timestamp, and Failure are stored as object (text) data.

Step 6: Obtain Dataset Information and Statistical Summary

```
print(df.info())
print(df.describe())
```

Step 7: Identify Missing Values

Missing values are common in real-world sensor data — for example, a tower may momentarily fail to transmit a reading.

```
print(df.isnull().sum())
```

To calculate the percentage of missing values in each column:

```
missing_percentage = df.isnull().mean() * 100
print(missing_percentage)
```

Expected Output: The dataset contains a small number of missing sensor readings — typically one or two — in columns such as Battery_Voltage_V and Signal_Strength_dBm.

Step 8: Handling Missing Values

Several common approaches exist for handling missing values. The choice of method depends on the nature of the column.

Method 1: Median Imputation (for numerical sensor readings)

Median is preferable to mean when the data may contain outliers, since it is not affected by extreme values:

```
df["Battery_Voltage_V"] = df["Battery_Voltage_V"].fillna(  
    df["Battery_Voltage_V"].median()  
df["Signal_Strength_dBm"] = df["Signal_Strength_dBm"].fillna(  
    df["Signal_Strength_dBm"].median())
```

Method 2: Mode Imputation (for categorical columns)

Mode imputation is generally used for categorical variables such as Region:

```
df["Region"] = df["Region"].fillna(df["Region"].mode()[0])
```

Verify that missing values have been handled:

```
print(df.isnull().sum())
```

Step 9: Detect and Remove Duplicate Records

Duplicate records can distort statistical analysis and bias machine learning models.

```
print("Duplicate rows:", df.duplicated().sum())  
print(df[df.duplicated()])  
  
df = df.drop_duplicates()  
print("Duplicate rows after cleaning:", df.duplicated().sum())
```

Step 10: Identify and Correct Inconsistent Categorical Data

Real-world datasets may contain inconsistent text values — for example, "Urban", "urban", " Urban " may represent the same category but are stored differently due to manual data entry.

```
print(df["Region"].unique())
```

Correct text inconsistencies by converting to a consistent case and removing unnecessary spaces:

```
df["Region"] = df["Region"].str.strip().str.title()  
print(df["Region"].unique())
```

Step 11: Detect Invalid Numerical Values

Telecom equipment is typically rated to operate safely within a known temperature range, for example -20°C to 80°C. Readings outside this range should be investigated as potential sensor faults:

```
invalid_temp = df[
    (df["Temperature_C"] < -20) |
    (df["Temperature_C"] > 80)
]
print("Invalid temperature readings:", len(invalid_temp))
```

Step 12: Detect Outliers using the IQR Method

An outlier is an observation that is significantly different from the other observations. The Interquartile Range (IQR) method is a common technique for outlier detection: $IQR = Q3 - Q1$, Lower limit = $Q1 - 1.5(IQR)$, Upper limit = $Q3 + 1.5(IQR)$.

```
Q1 = df["Temperature_C"].quantile(0.25)
Q3 = df["Temperature_C"].quantile(0.75)
IQR = Q3 - Q1
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

outliers = df[
    (df["Temperature_C"] < lower_limit) |
    (df["Temperature_C"] > upper_limit)]
print("Number of outliers:", len(outliers))
print(outliers[["Tower_ID", "Temperature_C"]])
```

Visualizing outliers using a box plot:

```
plt.figure(figsize=(6,5))
plt.boxplot(df["Temperature_C"].dropna())
plt.ylabel("Temperature (°C)")
plt.title("Equipment Temperature — Outlier Detection")
plt.grid(True)
plt.show()
```

Outliers should not automatically be deleted. Students should first determine whether a flagged reading represents a measurement error, sensor fault, data-entry error, or a genuine extreme operating condition, before deciding whether to remove or retain it.

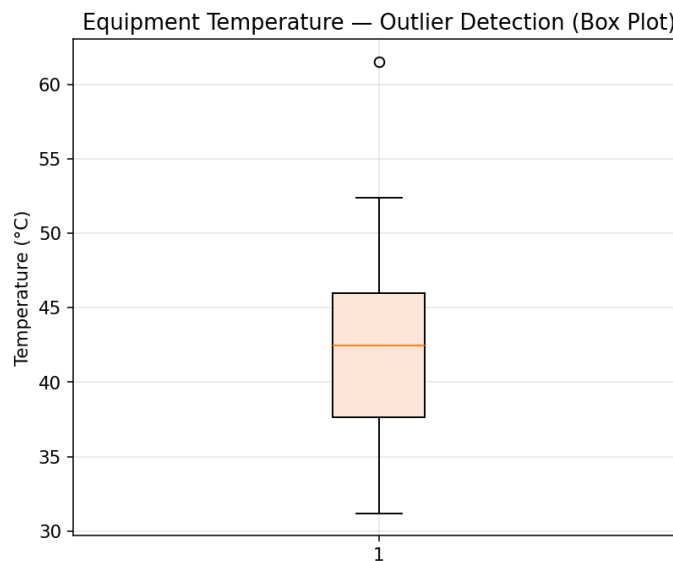


Fig. 3.2 — Actual box plot of equipment temperature; the dataset shows one mild high-temperature outlier

Step 13: Feature Selection

Feature selection means choosing the most relevant input variables for a machine learning model. It reduces computational cost, can improve model performance, and helps prevent overfitting.

```
features = ["Temperature_C", "Humidity_pct", "Battery_Voltage_V",
            "Current_A", "CPU_Load_pct", "Signal_Strength_dBm"]
X = df[features]
```

Step 14: Feature Engineering

Feature engineering involves creating useful new variables from existing data. Since Battery_Voltage_V and Current_A are available, an approximate instantaneous battery power draw can be engineered:

```
df["Battery_Power_W"] = df["Battery_Voltage_V"] * df["Current_A"]
print(df[["Battery_Voltage_V", "Current_A", "Battery_Power_W"]].head())
```

Step 15: Encoding Categorical Data

Machine learning algorithms generally require numerical input, so categorical columns such as Region and Failure must be converted into a numerical representation.

Label Encoding

Label encoding assigns an integer to each category. It is suitable for the binary target variable Failure:

```
encoder = LabelEncoder()
df["Failure_Encoded"] = encoder.fit_transform(df["Failure"])
print(df[["Failure", "Failure_Encoded"]].head())
```

One-Hot Encoding

One-hot encoding creates separate binary columns and is preferred for Region, since it has no natural order:

```
df = pd.get_dummies(df, columns=["Region"], dtype=int)
print(df.head())
```

Step 16: Save the Cleaned Dataset

The cleaned dataset is saved as a new CSV file and will be used as the starting point for feature scaling in Experiment No. 4.

```
df.to_csv("Cleaned_Telecom_Tower_Dataset.csv", index=False)
print("Cleaned dataset saved successfully.")
```

6. Result

The Telecom Tower IoT dataset was successfully loaded, inspected, and cleaned using Python and Pandas. Missing values were identified and imputed, duplicate records were checked and removed, inconsistent categorical text was standardized, outliers in equipment temperature were detected using the IQR method, a new engineered feature was created, categorical variables were encoded, and the cleaned dataset was saved for use in subsequent experiments.

7. Conclusion:

Date of Performance		Date of Submission	
Assessment Rubrics	Attendance	Performance (4)	Completion (3) Total (10)
Marks Obtained			
Teacher's Signature			

8. Viva Voce Questions

- Q1. Why is data preprocessing necessary before applying a Machine Learning algorithm?
- Q2. What is the difference between mean imputation and median imputation?
- Q3. How do you detect duplicate rows in a Pandas DataFrame?
- Q4. What is the IQR method for outlier detection?
- Q5. Should outliers always be deleted from a dataset?
- Q6. What is the difference between Label Encoding and One-Hot Encoding?
- Q7. Why was One-Hot Encoding chosen for the Region column instead of Label Encoding?
- Q8. What is feature engineering? Give an example from this experiment.
- Q9. What does `df.drop_duplicates()` do?
- Q10. Why is the cleaned dataset saved to a new CSV file rather than overwriting the original?

Experiment No. 4

Apply Feature Scaling, Normalization, and Standardization Techniques using Python Tools

1. Aim

To apply feature scaling techniques such as Min-Max Normalization, Standardization, and Robust Scaling on the cleaned Telecom Tower IoT dataset using Python and Scikit-learn, and to compare their effect on the numerical features.

2. Objectives

After completing this experiment, students will be able to:

- Understand why feature scaling is required before applying machine learning algorithms.
- Differentiate between normalization and standardization.
- Apply Min-Max Normalization using MinMaxScaler.
- Apply Standardization (Z-score scaling) using StandardScaler.
- Apply Robust Scaling using RobustScaler for outlier-prone features.
- Visualize and compare feature distributions before and after scaling.
- Select the appropriate scaling technique for a given engineering dataset.
- Save the scaled dataset for use in model-building experiments.

3. Software and Tools Required

Sr. No.	Software/Tool	Purpose
1	Python	Programming language
2	Google Colab / Jupyter Notebook	Python programming environment
3	Pandas	Dataset handling
4	NumPy	Numerical computation
5	Matplotlib	Visualization of feature distributions
6	Scikit-learn	MinMaxScaler, StandardScaler, RobustScaler
7	CSV Dataset	Cleaned_Telecom_Tower_Dataset.csv (from Experiment No. 3)

4. Theory

4.1 Why Feature Scaling is Required

Engineering datasets often contain features measured on very different numerical scales. In the Telecom Tower IoT dataset, for example, Battery_Voltage_V typically ranges between about 45–50 V, Current_A between about 3–5 A, while Cooling_Fan_RPM ranges into the thousands and Active_Users into the hundreds. Machine learning algorithms that rely on distance calculations (e.g., K-Means, K-Nearest Neighbors) or gradient-based optimization (e.g., Logistic Regression, Neural Networks) can be disproportionately influenced by features with larger numerical ranges unless all features are brought onto a comparable scale.

4.2 Min-Max Normalization

Min-Max normalization rescales each feature to a fixed range, commonly 0 to 1, using the formula:

$$X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Normalization preserves the shape of the original distribution but is sensitive to outliers, since a single extreme value compresses the rest of the data into a narrow sub-range.

4.3 Standardization (Z-score Scaling)

Standardization transforms data to have zero mean and unit standard deviation:

$$X_{\text{std}} = (X - \text{mean}) / \text{standard_deviation}$$

Standardization does not bound values to a fixed range and is generally preferred for algorithms that assume normally distributed data or use gradient-based optimization.

4.4 Robust Scaling

Robust scaling uses the median and the interquartile range (IQR) instead of the mean and standard deviation:

$$X_{\text{robust}} = (X - \text{median}) / \text{IQR}$$

Because it is based on the median and IQR rather than the mean and standard deviation, robust scaling is less influenced by outliers, making it well suited to sensor datasets that may contain occasional abnormal readings.

4.5 Comparison of Scaling Techniques

Feature	Min-Max Normalization	Standardization	Robust Scaling
Basis	Minimum and maximum	Mean and standard deviation	Median and IQR
Output range	Fixed (usually 0 to 1)	No fixed range	No fixed range
Sensitive to outliers	Yes — highly sensitive	Yes — moderately sensitive	No — robust to outliers
Typical use case	Neural networks, image data	Linear/Logistic Regression, PCA	Datasets with known outliers

4.6 Effect of Scaling on Algorithm Behaviour

Distance-based algorithms such as K-Means (Experiment No. 7) compute the Euclidean distance between observations across all selected features simultaneously. If Cooling_Fan_RPM (values in the thousands) is left unscaled alongside Current_A (values around 3–5), the distance calculation would be almost entirely dominated by Cooling_Fan_RPM, making the other features practically irrelevant to the clustering result. Similarly, gradient-based optimizers used to train Logistic Regression (Experiment No. 6) converge faster and more reliably when input features are on a comparable scale, since very large feature values can cause unstable or slow gradient updates. Scaling therefore directly affects both the correctness and the training efficiency of downstream models.

5. Procedure

Investigation 1: Examining Feature Ranges Before Scaling

Step 1: Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
```

Step 2: Load the Cleaned Dataset

```
df = pd.read_csv("Cleaned_Telecom_Tower_Dataset.csv")
print(df.head())
```

Step 3: Select Numerical Features for Scaling

```
num_features = ["Temperature_C", "Humidity_pct", "Battery_Voltage_V",
                "Current_A", "CPU_Load_pct", "Signal_Strength_dBm",
```

```
"Packet_Loss_pct", "Active_Users", "Power_Consumption_kW",  
"Cooling_Fan_RPM"]  
print(df[num_features].describe())
```

Step 4: Visualize Feature Ranges Before Scaling

```
plt.figure(figsize=(10,5))  
plt.boxplot(df[num_features].values, tick_labels=num_features)  
plt.title("Feature Ranges Before Scaling")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

Expected Output: The box plot shows that features such as Cooling_Fan_RPM and Active_Users occupy a much larger numerical range than features such as Current_A or Humidity_pct, confirming the need for scaling.

Investigation 2–4: Min-Max Normalization, Standardization, and Robust Scaling — Combined Comparison

```
minmax_scaler = MinMaxScaler()  
df_minmax = df.copy()  
df_minmax[num_features] = minmax_scaler.fit_transform(df[num_features])  
  
standard_scaler = StandardScaler()  
df_standard = df.copy()  
df_standard[num_features] = standard_scaler.fit_transform(df[num_features])  
  
robust_scaler = RobustScaler()  
df_robust = df.copy()  
df_robust[num_features] = robust_scaler.fit_transform(df[num_features])  
  
print(df_minmax[num_features].describe())  
print(df_standard[num_features].describe())  
print(df_robust[num_features].describe())
```

Expected Output: In df_minmax, every scaled column has minimum 0 and maximum 1. In df_standard, every scaled column has a mean close to 0 and a standard deviation close to 1. In df_robust, the median of every scaled column is close to 0.

```

fig, axes = plt.subplots(2, 2, figsize=(12, 9))
axes[0,0].boxplot(df[num_features].values, tick_labels=num_features)
axes[0,0].set_title("Before Scaling")
axes[0,1].boxplot(df_minmax[num_features].values, tick_labels=num_features)
axes[0,1].set_title("After Min-Max Normalization")
axes[1,0].boxplot(df_standard[num_features].values, tick_labels=num_features)
axes[1,0].set_title("After Standardization")
axes[1,1].boxplot(df_robust[num_features].values, tick_labels=num_features)
axes[1,1].set_title("After Robust Scaling")
for ax in axes.flat:
    ax.tick_params(axis="x", rotation=45)
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

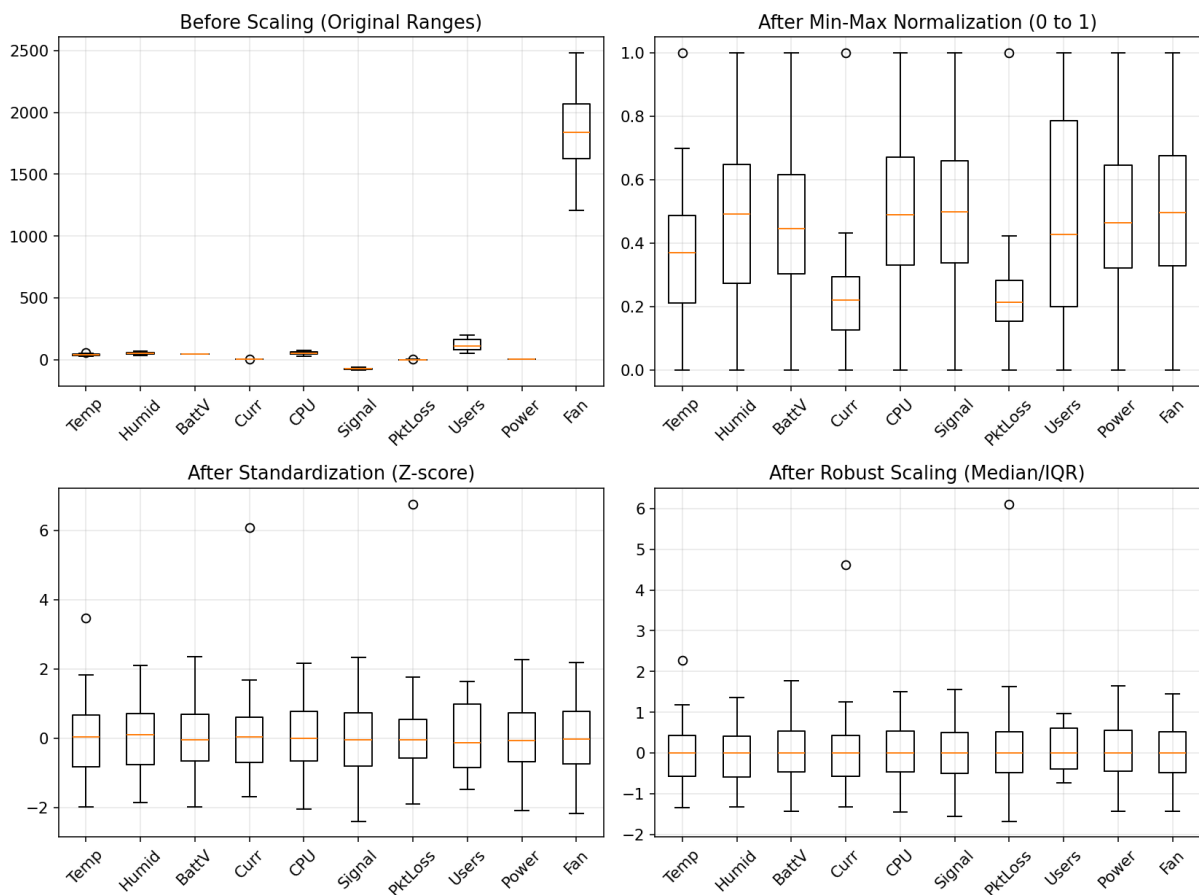


Fig. 4.1 — Actual feature distributions before scaling and after Min-Max Normalization, Standardization, and Robust Scaling, computed on the Telecom Tower IoT dataset

Observed Output: Before scaling, features such as Cooling_Fan_RPM and Power_Consumption_kW visually dominate the plot due to their large numerical range. After Min-Max Normalization, all

features are compressed into the 0–1 range. After Standardization, all features are centred around 0 with comparable spread. Robust Scaling produces a similar centred effect while remaining less sensitive to the single mild outlier identified in Experiment No. 3.

Investigation 5: Comparing a Single Feature Across All Three Techniques

```
comparison = pd.DataFrame({
    "Original": df["Temperature_C"],
    "MinMax": df_minmax["Temperature_C"],
    "Standardized": df_standard["Temperature_C"],
    "Robust": df_robust["Temperature_C"]
})
print(comparison.head(10))
```

Investigation 6: Saving the Scaled Dataset

For subsequent experiments, the standardized version of the dataset is saved, since Standardization is generally the recommended default for the regression, classification, clustering, and PCA experiments that follow.

```
df_standard.to_csv("Scaled_Telecom_Tower_Dataset.csv", index=False)
print("Scaled dataset saved successfully.")
```

6. Result

Feature scaling was successfully applied to the cleaned Telecom Tower IoT dataset using Min-Max Normalization, Standardization, and Robust Scaling. The effect of each technique on the numerical feature ranges was visualized and compared using box plots, and the standardized dataset was saved for use in subsequent machine learning experiments.

7. Conclusion:

Date of Performance		Date of Submission	
Assessment Rubrics	Attendance	Performance (4)	Completion (3) Total (10)
Marks Obtained			
Teacher's Signature			

8. Viva Voce Questions

- Q1. Why is feature scaling required before training a Machine Learning model?
- Q2. What is the formula for Min-Max Normalization?
- Q3. What is the formula for Standardization?
- Q4. What is the key difference between Standardization and Robust Scaling?
- Q5. Which scaling technique would you prefer for a dataset with several extreme outliers?
- Q6. Does Min-Max Normalization change the shape of the original data distribution?
- Q7. Which Scikit-learn class is used to perform Standardization?
- Q8. Why does K-Means clustering specifically require scaled input features?
- Q9. What does `fit_transform()` do, as opposed to `transform()` alone?
- Q10. After scaling, what should the mean and standard deviation of a standardized column be, approximately?

Experiment No. 5

Implement Linear Regression Algorithm for Prediction of Engineering Parameters

1. Aim

To implement the Linear Regression algorithm using Python and Scikit-learn for predicting engineering parameters from the Telecom Tower IoT dataset, and to evaluate the performance of the trained regression model using standard evaluation metrics.

2. Objectives

After completing this experiment, students will be able to:

- Understand the concept of Linear Regression and its use in engineering prediction problems.
- Differentiate between Simple Linear Regression and Multiple Linear Regression.
- Split the dataset into training and testing sets.
- Train a Linear Regression model using Scikit-learn.
- Predict engineering parameter values using the trained model.
- Visualize the regression line and residuals.
- Evaluate model performance using MAE, MSE, RMSE, and R^2 Score.
- Interpret the regression coefficients and intercept in an engineering context.

3. Software and Tools Required

Sr. No.	Software/Tool	Purpose
1	Python	Programming language
2	Google Colab / Jupyter Notebook	Python programming environment
3	Pandas	Dataset handling and analysis
4	NumPy	Numerical computation
5	Matplotlib	Data visualization
6	Scikit-learn	Model building and evaluation
7	CSV Dataset	Telecom_Tower_IoT_Dataset.csv

4. Theory

4.1 Introduction to Linear Regression

Linear Regression is a supervised Machine Learning algorithm used to model the relationship between one or more independent (input) variables and a continuous dependent (output) variable. It assumes that a linear relationship exists between the input features and the target variable.

In engineering applications, Linear Regression is widely used for prediction tasks such as estimating power consumption, predicting equipment temperature, forecasting signal degradation, and estimating battery voltage based on operating conditions.

4.2 Simple Linear Regression

Simple Linear Regression models the relationship between a single independent variable (X) and a dependent variable (Y) using a straight line equation:

$$Y = mX + c$$

Where Y is the predicted variable, X is the independent variable, m is the slope (coefficient), and c is the intercept. The algorithm determines the best-fit line by minimizing the sum of squared differences between the actual and predicted values, known as the Least Squares Method.

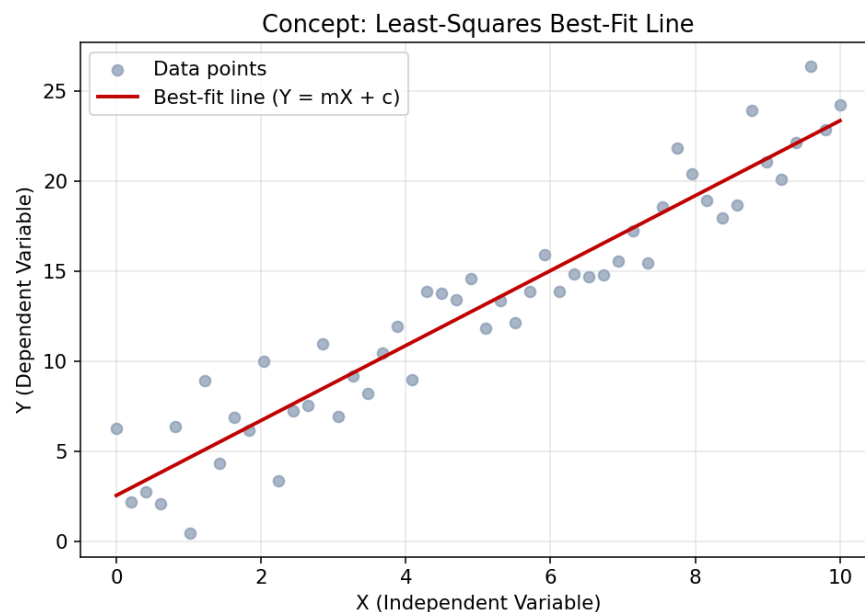


Fig. 5.1 — Concept of the least-squares best-fit line for Simple Linear Regression

4.3 The Least Squares Method

For a set of n data points, the Least Squares Method chooses the slope m and intercept c that minimize the Sum of Squared Errors (SSE) between the actual values y_i and the predicted values $\hat{y}_i$:

$$SSE = \sum (y_i - \hat{y}_i)^2, \quad i = 1 \text{ to } n$$

Because the errors are squared before summing, this method penalizes large deviations more heavily than small ones, and it has a unique closed-form solution, which is what makes ordinary Linear Regression fast and deterministic to train.

4.4 Multiple Linear Regression

Multiple Linear Regression extends the concept to more than one independent variable:

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

Where b_0 is the intercept and $b_1, b_2, \dots, b_n$ are the coefficients corresponding to each input feature. This is useful when the target variable depends on multiple engineering parameters simultaneously, such as predicting Power Consumption from Temperature, CPU Load, and Current.

4.5 Model Evaluation Metrics

Metric	Description
Mean Absolute Error (MAE)	Average of the absolute differences between actual and predicted values
Mean Squared Error (MSE)	Average of the squared differences between actual and predicted values
Root Mean Squared Error (RMSE)	Square root of MSE; expressed in the same unit as the target variable
R^2 Score	Indicates how well the independent variable(s) explain the variance in the target variable (closer to 1 is better)

MAE and RMSE are both expressed in the same physical unit as the target variable (kW, in this experiment), making them directly interpretable by an engineer, whereas R^2 is a unit-free measure of how much better the model performs compared to simply predicting the mean of the target for every observation. An R^2 close to 1 indicates that the model explains almost all the variability in the data; an R^2 close to 0 indicates the model performs no better than predicting the mean.

Note: This experiment loads the raw dataset directly for clarity. Students may alternatively use `Cleaned_Telecom_Tower_Dataset.csv` or `Scaled_Telecom_Tower_Dataset.csv` produced in Experiments No. 3 and 4.

5. Procedure

Investigation 1: Simple Linear Regression — Predicting Power Consumption from Temperature

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Step 2: Load the Dataset

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
print(df.head())
```

Step 3: Select Input Feature and Target Variable

```
X = df[["Temperature_C"]]
y = df["Power_Consumption_kW"]
```

Step 4: Split the Dataset into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

Step 5: Create and Train the Linear Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

Step 6: Display Model Parameters

```
print("Coefficient (Slope):", model.coef_[0])
print("Intercept:", model.intercept_)
```

Step 7: Predict on the Test Set

```
y_pred = model.predict(X_test)
print(y_pred[:5])
```

Step 8: Evaluate the Model

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
```

Step 9: Visualize the Regression Line

```
plt.figure(figsize=(8,5))
plt.scatter(X_test, y_test, color="blue", label="Actual")
plt.plot(X_test, y_pred, color="red", linewidth=2, label="Predicted (Regression Line)")
plt.title("Temperature vs Power Consumption — Linear Regression")
plt.xlabel("Temperature (°C)")
plt.ylabel("Power Consumption (kW)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

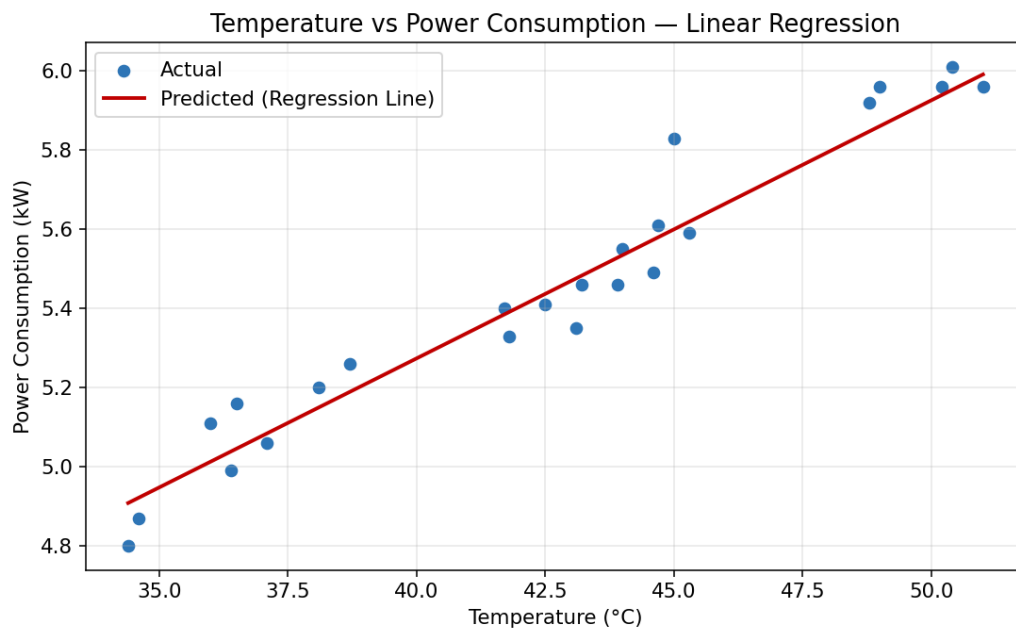


Fig. 5.2 — Actual regression line fitted on the test set (Temperature vs Power Consumption)

Step 10: Visualize the Residuals

```
residuals = y_test - y_pred
plt.figure(figsize=(8,5))
plt.scatter(y_pred, residuals, color="green")
plt.axhline(y=0, color="red", linestyle="--")
```

```
plt.title("Residual Plot")
plt.xlabel("Predicted Power Consumption (kW)")
plt.ylabel("Residuals")
plt.grid(alpha=0.3)
plt.show()
```

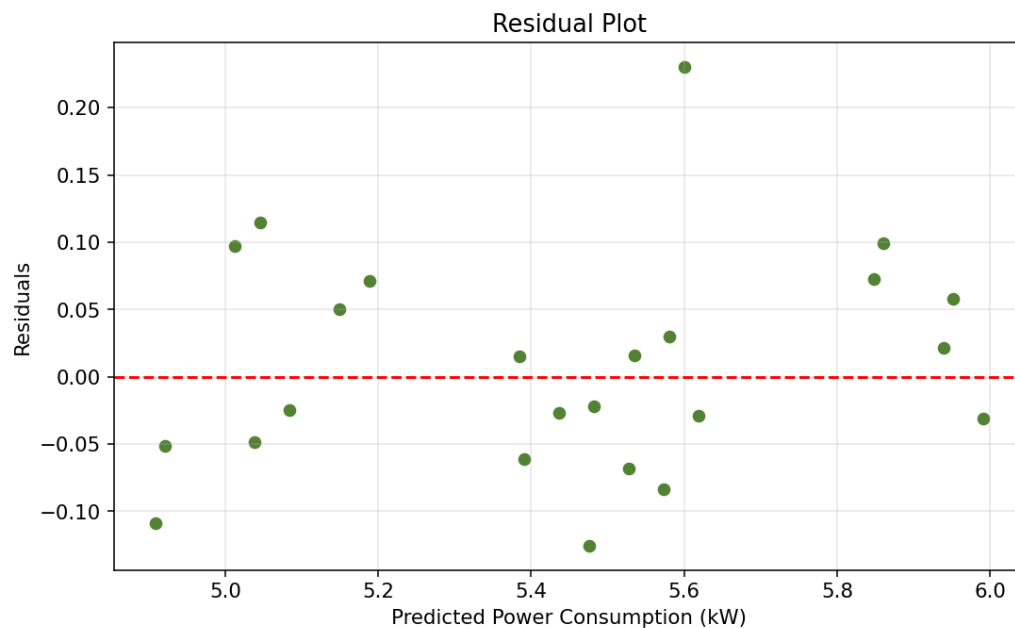


Fig. 5.3 — Actual residual plot for the trained model

Students should observe whether the residuals are randomly scattered around zero, which indicates a good linear fit, or whether they show a pattern, which indicates that the linear model may not be fully appropriate.

Investigation 2: Multiple Linear Regression — Predicting Power Consumption from Multiple Parameters

Step 1: Select Multiple Input Features

```
features = ["Temperature_C", "CPU_Load_pct", "Current_A", "Active_Users"]
X = df[features]
y = df["Power_Consumption_kW"]
```

Step 2: Split the Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)
```

Step 3: Train the Multiple Linear Regression Model

```
model_multi = LinearRegression()
model_multi.fit(X_train, y_train)
```

Step 4: Display Coefficients for Each Feature

```
coefficients = pd.DataFrame({
    "Feature": features,
    "Coefficient": model_multi.coef_
})
print(coefficients)
print("Intercept:", model_multi.intercept_)
```

Step 5: Predict and Evaluate the Multiple Regression Model

```
y_pred_multi = model_multi.predict(X_test)

mae = mean_absolute_error(y_test, y_pred_multi)
mse = mean_squared_error(y_test, y_pred_multi)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_multi)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
```

Step 6: Visualize Actual vs Predicted Values

```
plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred_multi, color="purple", alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color="red", linestyle="--")
plt.title("Actual vs Predicted Power Consumption")
plt.xlabel("Actual Power Consumption (kW)")
plt.ylabel("Predicted Power Consumption (kW)")
plt.grid(alpha=0.3)
plt.show()
```

Step 7: Compare Simple vs Multiple Linear Regression

Students should tabulate and compare the R^2 Score, MAE, and RMSE obtained from the Simple Linear Regression model (Investigation 1) and the Multiple Linear Regression model (Investigation 2), and analyze which model provides better predictive performance for Power Consumption.

Model	Input Feature(s)	R ² Score	RMSE
Simple Linear Regression	Temperature_C		
Multiple Linear Regression	Temperature_C, CPU_Load_pct, Current_A, Active_Users		

6. Result

A Linear Regression model was successfully implemented using Python and Scikit-learn to predict engineering parameters (Power Consumption) from the Telecom Tower IoT dataset. Both Simple Linear Regression (using a single feature) and Multiple Linear Regression (using multiple features) were trained, evaluated, and compared using MAE, MSE, RMSE, and R² Score, and the regression outputs were visualized using scatter plots, regression lines, and residual plots.

7. Conclusion:

Date of Performance		Date of Submission	
Assessment Rubrics	Attendance	Performance (4)	Completion (3) Total (10)
Marks Obtained			
Teacher's Signature			

8. Viva Voce Questions

Q1. What type of Machine Learning problem is Linear Regression used for?

Q2. What is the equation of Simple Linear Regression?

Q3. How does Linear Regression find the best-fit line?

Q4. What is the difference between Simple and Multiple Linear Regression?

Q5. What does the R^2 Score represent?

Q6. What is the difference between MSE and RMSE?

Q7. Why is the dataset split into training and testing sets?

Q8. What does the model's `coef_` attribute represent?

Q9. What should an ideal residual plot look like?

Q10. In this experiment, does Multiple Linear Regression outperform Simple Linear Regression?
How do you know?

Experiment No. 6

Develop Classification Models using Logistic Regression for Real-World Datasets

1. Aim

To develop a classification model using Logistic Regression on the Telecom Tower IoT dataset to predict equipment failure status, and to evaluate the model using standard classification metrics.

2. Objectives

After completing this experiment, students will be able to:

- Understand the concept of classification and how it differs from regression.
- Understand the Logistic Regression algorithm and the sigmoid function.
- Prepare a dataset for a classification task, including encoding the target variable.
- Train a Logistic Regression model using Scikit-learn.
- Predict class labels and class probabilities on unseen data.
- Evaluate a classification model using a confusion matrix, accuracy, precision, recall, and F1-score.
- Plot and interpret a Receiver Operating Characteristic (ROC) curve.

3. Software and Tools Required

Sr. No.	Software/Tool	Purpose
1	Python	Programming language
2	Google Colab / Jupyter Notebook	Python programming environment
3	Pandas	Dataset handling and analysis
4	NumPy	Numerical computation
5	Matplotlib	Data visualization
6	Scikit-learn	Model building and evaluation
7	CSV Dataset	Telecom_Tower_IoT_Dataset.csv

4. Theory

4.1 Classification vs Regression

While Linear Regression (Experiment No. 5) predicts a continuous numerical value, classification predicts a discrete category or class label. In this experiment, the target variable Failure (Yes/No) is a binary categorical variable, making this a binary classification problem.

4.2 Logistic Regression and the Sigmoid Function

Despite its name, Logistic Regression is a classification algorithm. It applies the sigmoid (logistic) function to the output of a linear equation to convert it into a probability between 0 and 1:

$$P(Y=1) = 1 / (1 + e^{-(b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n)})$$

The sigmoid function has a characteristic S-shape: it maps any real-valued input to an output strictly between 0 and 1, approaching 0 for large negative inputs and approaching 1 for large positive inputs, with the steepest change occurring near an input of zero.

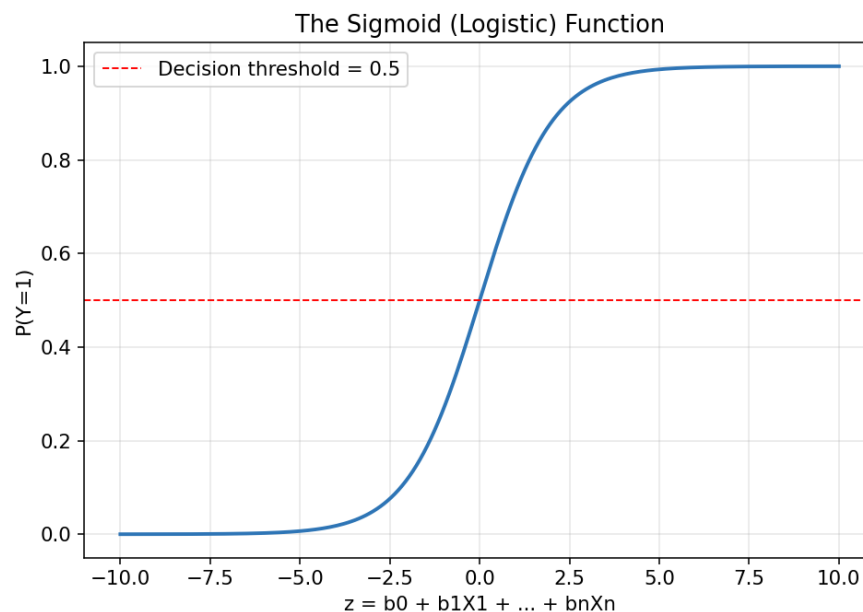


Fig. 6.1 — The sigmoid function used by Logistic Regression to map a linear score to a probability

If the predicted probability is greater than a chosen threshold (commonly 0.5), the observation is classified into the positive class (Failure = Yes); otherwise it is classified into the negative class (Failure = No). The threshold itself is a tunable design choice: lowering it increases sensitivity to the positive class (more predicted failures, at the cost of more false alarms), while raising it does the opposite.

4.3 Class Imbalance

In the Telecom Tower IoT dataset, equipment failure is a rare event — the large majority of towers report Failure = No, with only a small number of Failure = Yes records. This is known as class

imbalance and is common in real-world fault-detection problems. Special care is required when evaluating such models, since a model that always predicts the majority class can still show high accuracy while being practically useless. Techniques such as `class_weight="balanced"` and stratified train-test splitting help address this.

4.4 The Confusion Matrix

A confusion matrix cross-tabulates predicted labels against actual labels for a classification problem, and is the basis for every other classification metric used in this experiment:

	Predicted: No Failure	Predicted: Failure
Actual: No Failure	True Negative (TN)	False Positive (FP)
Actual: Failure	False Negative (FN)	True Positive (TP)

In a predictive-maintenance context, a False Negative (a real failure predicted as “No Failure”) is generally far more costly than a False Positive (an unnecessary inspection), since it means a genuine fault goes undetected — which is why Recall is often prioritized over plain Accuracy for this type of problem.

4.5 Classification Evaluation Metrics

Metric	Formula	Description
Accuracy	$(TP+TN) / (TP+TN+FP+FN)$	Proportion of total predictions that are correct
Precision	$TP / (TP+FP)$	Of all predicted failures, the proportion that were actual failures
Recall (Sensitivity)	$TP / (TP+FN)$	Of all actual failures, the proportion correctly identified
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Harmonic mean of Precision and Recall
ROC-AUC	Area under the ROC curve	Measures separability between classes across all thresholds

5. Procedure

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score,
                             recall_score, f1_score, classification_report,
                             roc_curve, roc_auc_score)

```

Step 2: Load the Dataset

```

df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
print(df.head())

```

Step 3: Encode the Target Variable

```

encoder = LabelEncoder()
df["Failure_Encoded"] = encoder.fit_transform(df["Failure"]) # No=0, Yes=1
print(df["Failure_Encoded"].value_counts())

```

Step 4: Select Input Features and Target

```

features = ["Temperature_C", "Battery_Voltage_V", "CPU_Load_pct",
            "Signal_Strength_dBm", "Packet_Loss_pct"]
X = df[features]
y = df["Failure_Encoded"]

```

Step 5: Split the Dataset (Stratified)

Since failure cases are rare, stratified splitting is used to ensure both the training and testing sets contain a representative proportion of each class:

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

```

Step 6: Scale the Input Features

Logistic Regression is a gradient-based algorithm and benefits from standardized input features:

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

Step 7: Train the Logistic Regression Model

```

model = LogisticRegression(class_weight="balanced", random_state=42)
model.fit(X_train_scaled, y_train)

```

Step 8: Predict Class Labels and Probabilities

```

y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[: , 1]
print(y_pred)

```

Step 9: Generate the Confusion Matrix

```

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

plt.figure(figsize=(5,5))
plt.imshow(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.colorbar()
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j], ha="center", va="center", color="black")
plt.show()

```

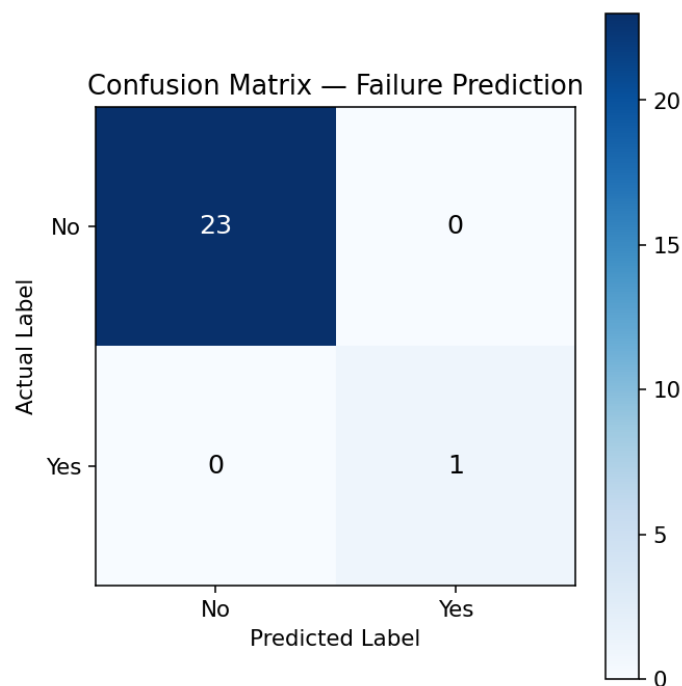


Fig. 6.2 — Actual confusion matrix from the trained Logistic Regression model on the test set

Step 10: Compute Classification Metrics

```

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision :", precision_score(y_test, y_pred, zero_division=0))
print("Recall   :", recall_score(y_test, y_pred, zero_division=0))

```

```
print("F1 Score :", f1_score(y_test, y_pred, zero_division=0))
print(classification_report(y_test, y_pred, zero_division=0))
```

Step 11: Plot the ROC Curve

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, color="darkorange", label=f"ROC Curve (AUC = {auc:.2f})")
plt.plot([0, 1], [0, 1], color="grey", linestyle="--")
plt.title("ROC Curve — Failure Prediction")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

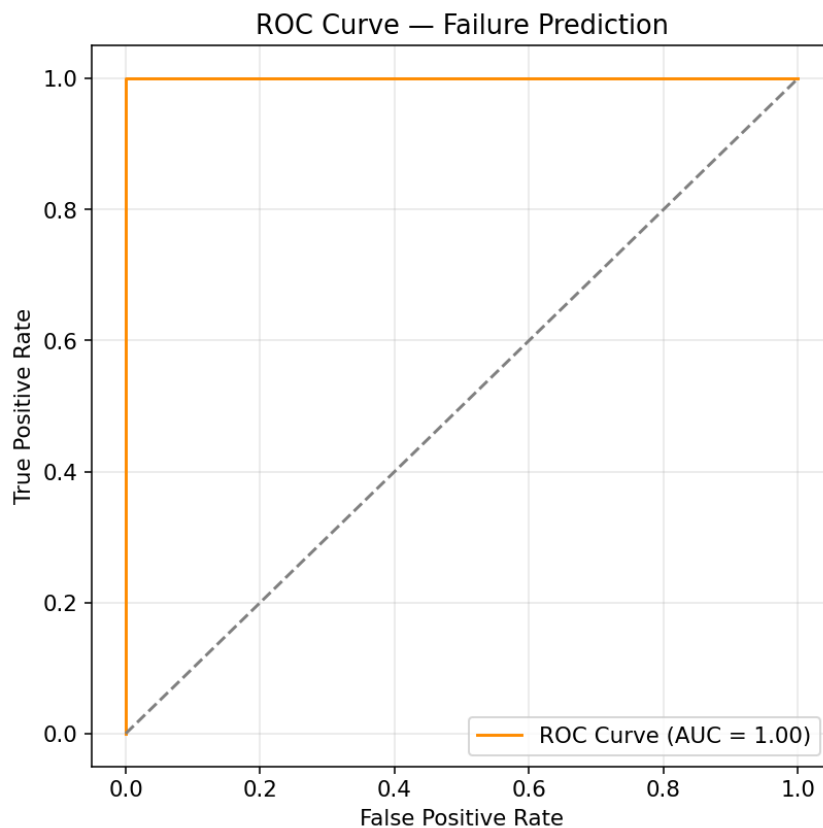


Fig. 6.3 — Actual ROC curve for the trained model

Expected Output: An ROC curve bowed toward the top-left corner and an AUC value greater than 0.5 indicate that the model separates failure and non-failure cases better than random guessing.

Step 12: Interpret the Model Coefficients

```
coefficients = pd.DataFrame({
    "Feature": features,
    "Coefficient": model.coef_[0]
})
print(coefficients)
```

A positive coefficient indicates that an increase in that feature increases the predicted probability of failure, while a negative coefficient indicates the opposite, consistent with the engineering relationships identified in Experiment No. 2.

6. Result

A Logistic Regression classification model was successfully developed to predict telecom tower equipment failure using Python and Scikit-learn. The model was trained on standardized features, evaluated using a confusion matrix, accuracy, precision, recall, F1-score, and the ROC-AUC curve, and the influence of individual engineering parameters on the predicted failure probability was interpreted.

7. Conclusion:

Date of Performance			Date of Submission		
Assessment Rubrics	Attendance	Performance (4)	Completion (3)	Total (10)	
Marks Obtained					
Teacher's Signature					

8. Viva Voce Questions

- Q1. Why is Logistic Regression used for classification and not regression, despite its name?
- Q2. What is the range of output values produced by the sigmoid function?
- Q3. What is the default decision threshold used to convert a predicted probability into a class label?
- Q4. What is class imbalance, and why is it relevant to this dataset?
- Q5. Why is accuracy alone a misleading metric for an imbalanced dataset?
- Q6. Define Precision and Recall in the context of failure prediction.
- Q7. In predictive maintenance, why might Recall be more important than Precision?
- Q8. What does the ROC-AUC value represent?
- Q9. Why was `class_weight="balanced"` used when training the model?
- Q10. What does a positive coefficient for `CPU_Load_pct` mean in this model?

Experiment No. 7

Implement K-Means Clustering Algorithm for Grouping and Pattern Analysis of Datasets

1. Aim

To implement the K-Means clustering algorithm using Python and Scikit-learn to group telecom towers with similar operating characteristics and to analyze the resulting patterns.

2. Objectives

After completing this experiment, students will be able to:

- Understand the concept of unsupervised learning and clustering.
- Understand how the K-Means algorithm assigns observations to clusters.
- Determine a suitable number of clusters using the Elbow Method.
- Train a K-Means clustering model using Scikit-learn.
- Visualize the resulting clusters using a scatter plot.
- Evaluate clustering quality using the Silhouette Score.
- Interpret the operating characteristics of each cluster in an engineering context.

3. Software and Tools Required

Sr. No.	Software/Tool	Purpose
1	Python	Programming language
2	Google Colab / Jupyter Notebook	Python programming environment
3	Pandas	Dataset handling and analysis
4	NumPy	Numerical computation
5	Matplotlib	Data visualization
6	Scikit-learn	KMeans, StandardScaler, silhouette_score
7	CSV Dataset	Telecom_Tower_IoT_Dataset.csv

4. Theory

4.1 Unsupervised Learning

Unlike Linear Regression and Logistic Regression, which require a labelled target variable, unsupervised learning algorithms find structure in data without predefined labels. K-Means Clustering is one of the most widely used unsupervised learning algorithms, used to group similar observations together based on their feature values.

4.2 The K-Means Algorithm

K-Means partitions the dataset into K clusters by iteratively performing the following steps:

- Randomly initialize K cluster centroids.
- Assign each observation to the nearest centroid based on Euclidean distance.
- Recompute each centroid as the mean of the observations assigned to it.
- Repeat the assignment and update steps until the centroids no longer change significantly.

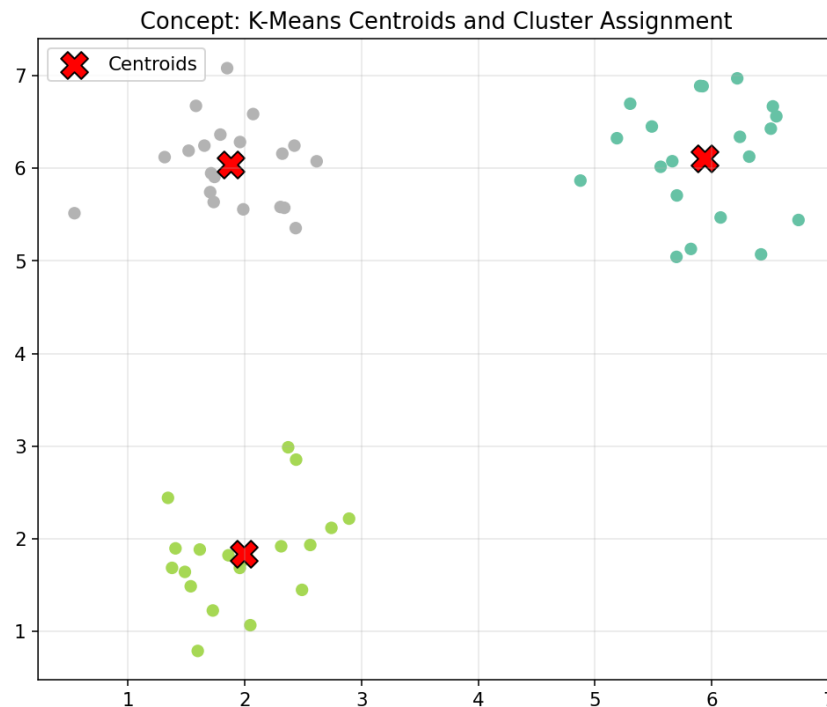


Fig. 7.1 — Concept illustration: K-Means centroids (red X) and the resulting cluster assignment on a sample 2D dataset

4.3 Choosing the Number of Clusters (Elbow Method)

The Within-Cluster Sum of Squares (WCSS), also called inertia, measures the compactness of the clusters — specifically, the sum of squared distances between each point and its assigned centroid.

WCSS always decreases as K increases (more clusters can only fit the data at least as well), so it cannot be minimized directly. The Elbow Method instead plots WCSS against different values of K; the point where the rate of decrease sharply slows (the “elbow”) is generally chosen as a good trade-off between model simplicity and cluster compactness.

4.4 Silhouette Score

The Silhouette Score measures how similar an observation is to its own cluster compared to other clusters, combining both cohesion (closeness within a cluster) and separation (distance from other clusters) into a single number. It ranges from -1 to +1, where values closer to +1 indicate well-separated, well-defined clusters, values near 0 indicate overlapping clusters, and negative values suggest that observations may have been assigned to the wrong cluster.

5. Procedure

Step 1: Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

Step 2: Load the Dataset

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
print(df.head())
```

Step 3: Select Features for Clustering

Towers are grouped based on their thermal and power-related operating characteristics:

```
features = ["Temperature_C", "CPU_Load_pct", "Power_Consumption_kW"]
X = df[features]
```

Step 4: Scale the Features

K-Means relies on Euclidean distance, so all features must be scaled to a comparable range before clustering:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 5: Apply the Elbow Method

```
wcss = []
K_range = range(1, 11)
for k in K_range:
```

```

kmeans = KMeans(n_clusters=k, init="k-means++", random_state=42, n_init=10)
kmeans.fit(X_scaled)
wcss.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(K_range, wcss, marker="o")
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS (Inertia)")
plt.grid(alpha=0.3)
plt.show()

```

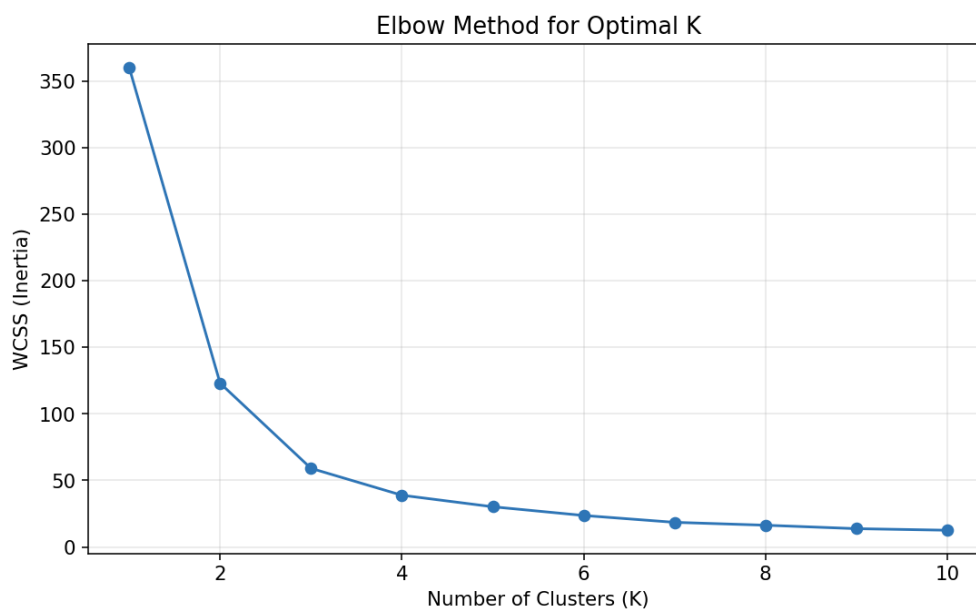


Fig. 7.2 — Actual Elbow Method plot computed on the Telecom Tower IoT dataset

Expected Output: A curve that decreases steeply for small K and flattens out beyond K = 3, suggesting three clusters is a reasonable choice — consistent with the three Region categories (Urban, Semi-Urban, Rural).

Step 6: Train the K-Means Model with the Chosen K

```

optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, init="k-means++", random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(X_scaled)
df["Cluster"] = cluster_labels
print(df[["Tower_ID", "Region", "Cluster"]].head(10))

```

Step 7: Visualize the Clusters

```
plt.figure(figsize=(8,6))
scatter = plt.scatter(df["CPU_Load_pct"], df["Temperature_C"],
                     c=df["Cluster"], cmap="viridis", alpha=0.8)
plt.title("Telecom Tower Clusters — CPU Load vs Temperature")
plt.xlabel("CPU Load (%)")
plt.ylabel("Temperature (°C)")
plt.colorbar(scatter, label="Cluster")
plt.grid(alpha=0.3)
plt.show()
```

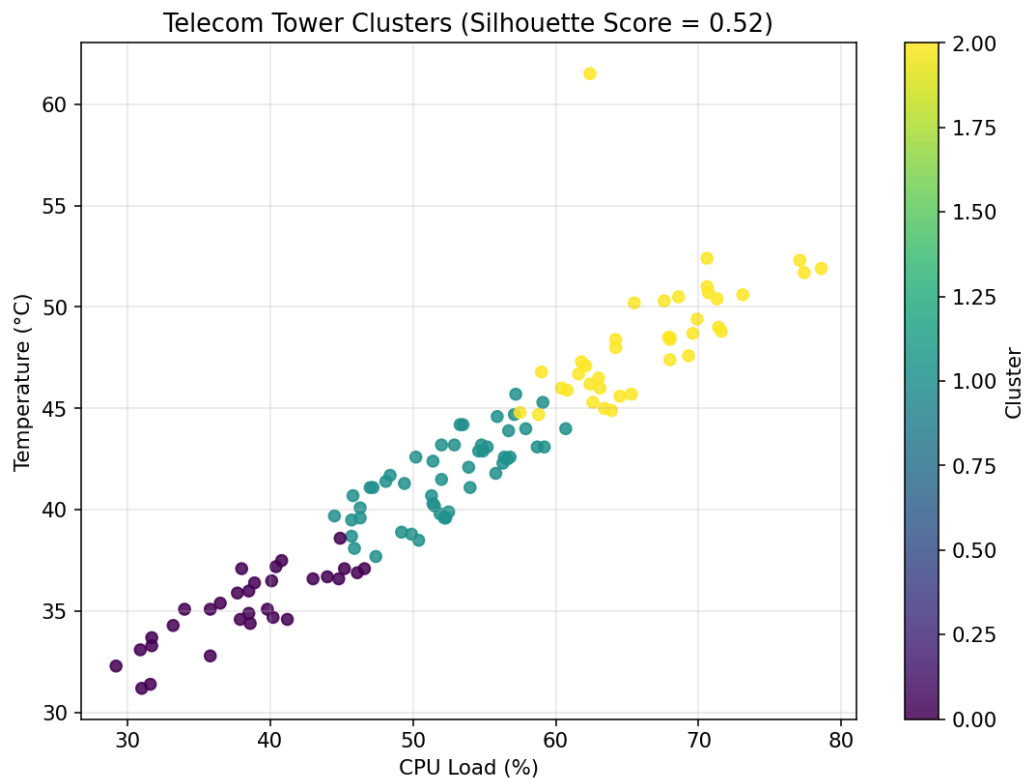


Fig. 7.3 — Actual K-Means clustering result (K=3) on the Telecom Tower IoT dataset, with the computed Silhouette Score

Step 8: Evaluate Clustering Quality

```
sil_score = silhouette_score(X_scaled, cluster_labels)
print("Silhouette Score:", sil_score)
```

Step 9: Analyze Cluster Characteristics

```
cluster_summary = df.groupby("Cluster")[features].mean()
print(cluster_summary)
```

Students should interpret each cluster in engineering terms — for example, identifying a cluster of towers operating with low temperature, low CPU load, and low power consumption (lightly loaded towers) versus a cluster with high values across all three (heavily loaded, thermally stressed towers).

Step 10: Compare Clusters with the Region Feature

```
comparison = pd.crosstab(df["Cluster"], df["Region"])
print(comparison)
```

This comparison helps students examine whether the clusters discovered purely from operating conditions align with the geographical Region labels, or whether they reveal a different underlying grouping.

6. Result

The K-Means clustering algorithm was successfully implemented on the Telecom Tower IoT dataset. The optimal number of clusters was determined using the Elbow Method, towers were grouped based on their thermal and power characteristics, the clusters were visualized and evaluated using the Silhouette Score, and the operating characteristics of each cluster were analyzed and compared with the Region feature.

7. Conclusion:

Date of Performance			Date of Submission		
Assessment Rubrics	Attendance	Performance (4)	Completion (3)	Total (10)	
Marks Obtained					
Teacher's Signature					

8. Viva Voce Questions

- Q1. Is K-Means a supervised or unsupervised learning algorithm?
- Q2. What does the 'K' in K-Means represent?
- Q3. Briefly describe the steps of the K-Means algorithm.
- Q4. What distance metric does K-Means typically use?
- Q5. Why must features be scaled before applying K-Means?
- Q6. What is WCSS (inertia), and how is it used in the Elbow Method?
- Q7. What does the Silhouette Score measure?
- Q8. What would a Silhouette Score close to 0 indicate?
- Q9. Why was $K = 3$ chosen for the Telecom Tower dataset?
- Q10. Does the cluster grouping obtained necessarily match the Region labels? Why or why not?

Experiment No. 8

Demonstrate Dimensionality Reduction using Principal Component Analysis (PCA)

1. Aim

To demonstrate dimensionality reduction on the Telecom Tower IoT dataset using Principal Component Analysis (PCA) with Python and Scikit-learn, and to visualize and interpret the reduced feature space.

2. Objectives

After completing this experiment, students will be able to:

- Understand the need for dimensionality reduction in engineering datasets.
- Understand the concept of principal components and explained variance.
- Standardize data before applying PCA.
- Apply PCA using Scikit-learn to reduce a multi-feature dataset to two dimensions.
- Plot and interpret a scree plot of explained variance.
- Visualize the reduced dataset and interpret the resulting structure.

3. Software and Tools Required

Sr. No.	Software/Tool	Purpose
1	Python	Programming language
2	Google Colab / Jupyter Notebook	Python programming environment
3	Pandas	Dataset handling and analysis
4	NumPy	Numerical computation
5	Matplotlib	Data visualization
6	Scikit-learn	StandardScaler, PCA
7	CSV Dataset	Telecom_Tower_IoT_Dataset.csv

4. Theory

4.1 The Need for Dimensionality Reduction

The Telecom Tower IoT dataset contains ten or more numerical engineering parameters. Working with many correlated features increases computational cost, makes visualization difficult, and can introduce redundancy, since parameters such as `Temperature_C`, `CPU_Load_pct`, and `Power_Consumption_kW` are known to be correlated. Dimensionality reduction techniques compress this information into a smaller number of new features while retaining as much of the original variability as possible.

4.2 Principal Component Analysis (PCA)

PCA transforms a set of possibly correlated features into a smaller set of uncorrelated variables called principal components. Each principal component is a linear combination of the original features, computed such that:

- The first principal component (PC1) captures the maximum possible variance in the data.
- The second principal component (PC2) captures the maximum remaining variance, subject to being uncorrelated (orthogonal) to PC1.
- Subsequent components continue this pattern, each capturing progressively less variance.

Mathematically, PCA is computed from the eigenvectors and eigenvalues of the covariance matrix of the standardized data: each eigenvector defines the direction of a principal component, and its corresponding eigenvalue is proportional to the amount of variance captured along that direction.

4.3 Explained Variance Ratio

The explained variance ratio indicates the proportion of the dataset's total variance captured by each principal component. Plotting cumulative explained variance against the number of components (a scree plot) helps determine how many components are needed to retain most of the information in the original dataset. A common rule of thumb is to retain enough components to explain 90–95% of the total variance.

4.4 Importance of Standardization

PCA is sensitive to the scale of the input features, since it is based on variance. Features with larger numerical ranges would otherwise dominate the principal components. All numerical features must therefore be standardized before applying PCA — this is the same `StandardScaler` technique introduced in Experiment No. 4.

5. Procedure

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Step 2: Load the Dataset

```
df = pd.read_csv("Telecom_Tower_IoT_Dataset.csv")
print(df.head())
```

Step 3: Select Numerical Features

```
features = ["Temperature_C", "Humidity_pct", "Battery_Voltage_V", "Current_A",
            "CPU_Load_pct", "Signal_Strength_dBm", "Packet_Loss_pct",
            "Active_Users", "Power_Consumption_kW", "Cooling_Fan_RPM"]
X = df[features]
```

Step 4: Standardize the Features

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 5: Apply PCA with All Components

PCA is first applied with all components to study how much variance each one explains:

```
pca_full = PCA()
pca_full.fit(X_scaled)

explained_variance = pca_full.explained_variance_ratio_
print("Explained Variance Ratio:", explained_variance)
```

Step 6: Plot the Scree Plot

```
plt.figure(figsize=(8,5))
plt.plot(range(1, len(explained_variance) + 1),
         np.cumsum(explained_variance), marker="o")
plt.title("Cumulative Explained Variance vs Number of Components")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
```

```
plt.grid(alpha=0.3)
plt.show()
```

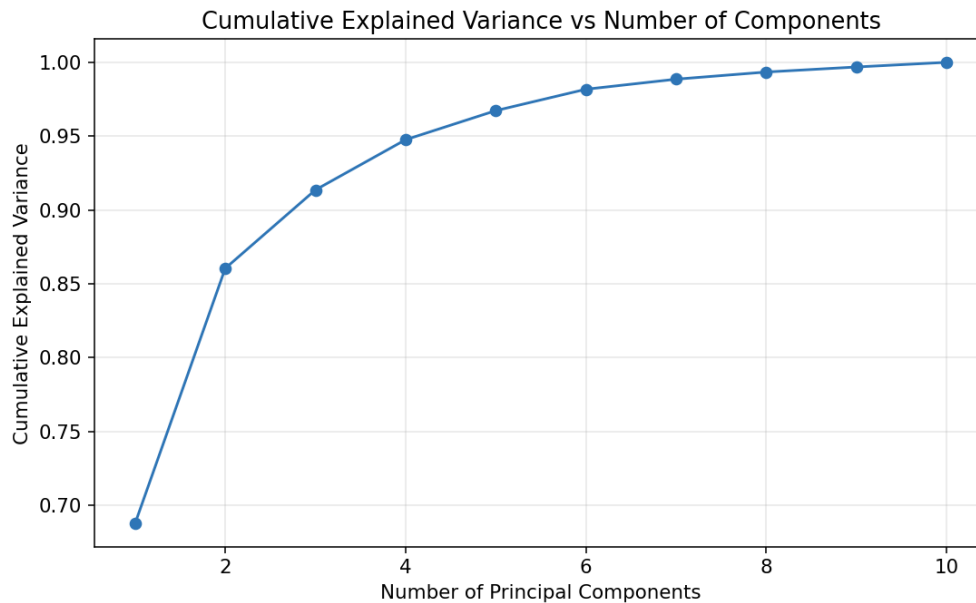


Fig. 8.1 — Actual scree plot (cumulative explained variance) computed on the Telecom Tower IoT dataset

Expected Output: The cumulative explained variance rises quickly for the first two to three components, confirming that a small number of principal components can represent most of the information present in the original ten engineering parameters.

Step 7: Reduce the Dataset to Two Principal Components

```
pca_2d = PCA(n_components=2)
principal_components = pca_2d.fit_transform(X_scaled)

df_pca = pd.DataFrame(principal_components, columns=["PC1", "PC2"])
df_pca["Region"] = df["Region"].values
df_pca["Failure"] = df["Failure"].values
print(df_pca.head())
```

Step 8: Visualize the Two-Dimensional PCA Projection by Region

```
plt.figure(figsize=(8,6))
for region in df_pca["Region"].unique():
    subset = df_pca[df_pca["Region"] == region]
    plt.scatter(subset["PC1"], subset["PC2"], label=region, alpha=0.8)

plt.title("PCA — Telecom Tower Dataset (2 Components)")
plt.xlabel(f"Principal Component 1 ({explained_variance[0]*100:.1f}% variance)")
```

```
plt.ylabel(f"Principal Component 2 ({explained_variance[1]*100:.1f}% variance)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

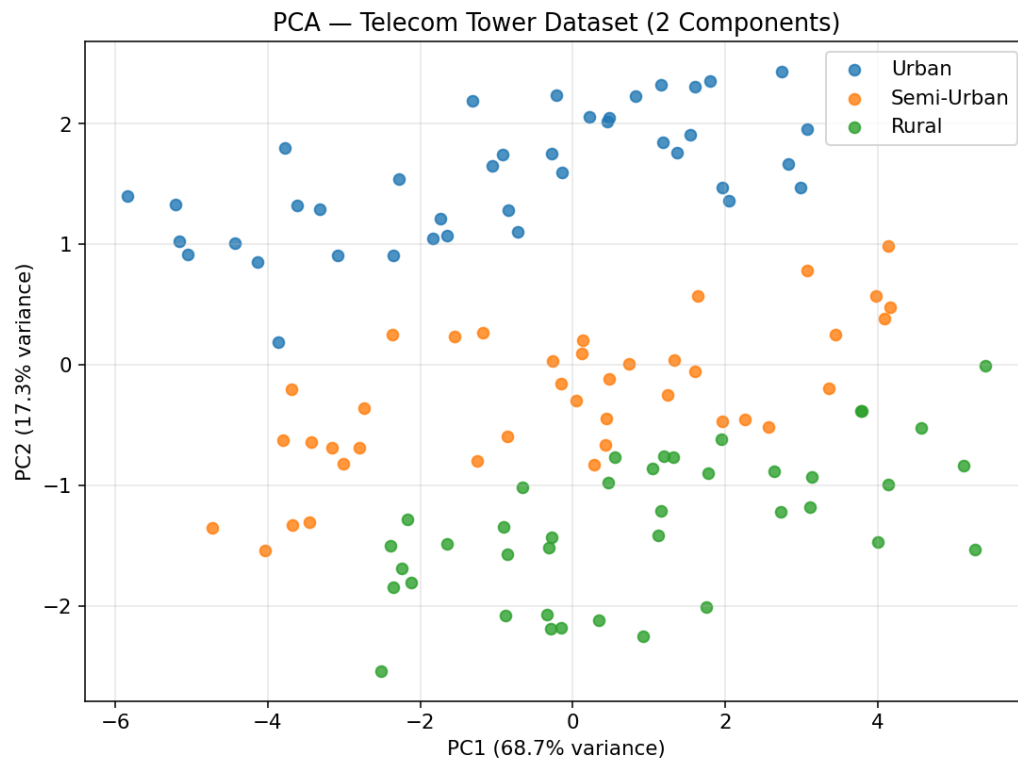


Fig. 8.2 — Actual 2D PCA projection of the Telecom Tower IoT dataset, coloured by Region

Step 9: Visualize the Two-Dimensional PCA Projection by Failure Status

```
plt.figure(figsize=(8,6))
for status in df_pca["Failure"].unique():
    subset = df_pca[df_pca["Failure"] == status]
    plt.scatter(subset["PC1"], subset["PC2"], label=f"Failure = {status}", alpha=0.8)

plt.title("PCA Projection Colored by Failure Status")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Students should observe whether towers with Failure = Yes tend to cluster in a distinct region of the PCA plot, which would suggest that the engineering parameters carry useful information for predicting failure, consistent with the classification model built in Experiment No. 6.

6. Result

Principal Component Analysis was successfully applied to the Telecom Tower IoT dataset to reduce ten standardized engineering features to two principal components. The explained variance of each component was analyzed using a scree plot, and the reduced two-dimensional representation of the dataset was visualized and interpreted with respect to Region and Failure status.

7. Conclusion:

Date of Performance		Date of Submission	
Assessment Rubrics	Attendance	Performance (4)	Completion (3) Total (10)
Marks Obtained			
Teacher's Signature			

8. Viva Voce Questions

- Q1. What is the main purpose of Principal Component Analysis (PCA)?
- Q2. Is PCA a supervised or unsupervised technique?
- Q3. Why must data be standardized before applying PCA?
- Q4. What does the first principal component (PC1) represent?
- Q5. What is 'explained variance ratio'?
- Q6. What is a scree plot used for?
- Q7. Are the principal components correlated with each other?

Q8. Can the principal components be directly interpreted as one original feature (e.g., Temperature)?

Q9. What is a typical engineering use of PCA before applying another Machine Learning model?

Q10. In this experiment, how many principal components were used for visualization, and why?